BVRIT HYDERABAD

College of Engineering for Women

Department of CSE (AI&ML)



CASE STUDY

Student Information System

Under the guidance of

Miss V. Asha (Asst. Professor)

TEAM MEMBERS

25WH1A6667	–	A. Sreeja
25WH1A6674	–	G. Pranathi
25WH1A6681	–	M. Neharika
25WH1A6688	–	K. Thanusree
25WH1A6695	–	V. Srinija
25WH1A66A2	–	Y. Bhuvani
25WH1A66B0	–	B. Samanvi
25WH1A66B7	–	A. Sucharitha
25WH1A66C4	–	Vibha Manomai
26WH5A6610	–	D. Vyshnavi

INDEX

S.NO.	TOPIC	PAGE NO.
1	**Introduction to DBMS and Project Overview**	**3**
1.1	Overview	3
1.2	Problem Statement	3
1.3	Objectives of the Project	3
1.4	Scope of the Project	3
1.5	Need and Importance	4
1.6	Proposed System Overview	4
1.7	Technologies Used	5
1.8	Conclusion	5
2	**Requirement Analysis**	**6**
2.1	Hardware Requirements	6
2.2	Software Requirements	6
2.3	Functional Requirements	6
2.4	Non-Functional Requirements	8
2.5	Integration Requirements	9
2.6	Project Scope and Feasibility	9
2.7	Conclusion	9
3	**ER Model and Database Design**	**10**
3.1	Entity Relationship Diagram	10
3.2	Entities and Key Identification	10
3.3	Relationships and Cardinalities	11
4	**Relational Model and Database Implementation**	**13**
4.1	Database Creation	13
4.2	Table Creation, Data Insertion and Verification	13
5	**SQL Fundamentals and Queries**	**23**
5.1	Data Definition Language (DDL)	23
5.2	Data Manipulation Language (DML)	24
5.3	Sorting and Aggregate Functions	26
5.4	Joins and Set Operations	28
5.5	GROUP BY and HAVING with Joins	31
5.6	Nested Queries and Subqueries	34
5.7	Stored Procedures	36
5.8	Triggers and Data Validation	41
6	**Views**	**49**
7	**Database Security and Transaction Management**	**55**
7.1	Data Control Language (DCL): GRANT and REVOKE	55
7.2	Transaction Control Language (TCL): COMMIT, ROLLBACK and SAVEPOINT	56
8	**Testing and Results**	**60**
9	**Conclusion**	**61**

1. Introduction to DBMS and Project Overview

1.1 Overview

A Student Information System (SIS) is a database-driven system used to organize and manage the academic information of an educational institution. This project demonstrates a MySQL relational database that stores student and academic records in structured tables and connects the related information through primary and foreign keys.

The database covers departments, students, faculty, courses, enrollments, attendance, examinations, marks, fees and user accounts. Keeping these records in related tables lets users retrieve information without repeatedly storing complete student, course or department details, and it gives a clear foundation for running SQL queries and preparing academic summaries.

1.2 Problem Statement

When academic records are kept in paper registers or in separate, disconnected files, finding a student's details can take a long time. The same information may be entered more than once, and an update made in one file may not appear in another. Tracking enrollment, attendance, examination marks and fee status becomes difficult, and finding related course, faculty or department information may require checking several records.

A relational database addresses these issues by organizing data into tables linked through primary and foreign keys. SQL queries retrieve and summarize the connected information, while constraints, views and triggers help maintain consistency, restrict access and validate data.

1.3 Objectives of the Project

The project aims to demonstrate how a relational database can support the organization and retrieval of student-related academic information. Its main objectives are:

1. Design a structured relational database for student-related academic information.
2. Store and retrieve student and department information efficiently.
3. Maintain faculty and course details and connect them through keys.
4. Record enrollment, attendance, examination and marks information.
5. Maintain fee and user-account records within the documented scope.
6. Demonstrate SQL data definition and data manipulation operations.
7. Use joins, aggregate functions and nested queries to retrieve and summarize data.
8. Demonstrate stored procedures, triggers and database constraints where appropriate.
9. Create views to simplify queries and restrict the data exposed to users.
10. Demonstrate DCL (GRANT, REVOKE) and TCL (COMMIT, ROLLBACK, SAVEPOINT) commands for access control and transaction management.

1.4 Scope of the Project

The project scope includes a ten-entity relational database, an ER diagram, sample data insertion and verification, and SQL demonstrations. The database covers department and student details; faculty and course information; student enrollment; attendance; examinations and marks; fee records; and user-account records. The documented SQL work includes data definition and manipulation, sorting and aggregate functions, joins, nested queries, stored procedures, triggers, views, data control and transaction control.

This is a database-focused academic project rather than a complete college management application. It does not include a fully developed website, a functioning login interface, online fee payments, a mobile application or biometric attendance. User-account and password-hash values are demonstration data.

1.5 Need and Importance

A centralized relational database helps organize academic records and makes it easier to retrieve, update and summarize information. Related data can be combined when needed instead of being copied into several files. This reduces avoidable duplication and supports reports such as department-wise student counts, course-wise marks summaries and attendance lookups.

A **primary key** uniquely identifies a row in a table. A **foreign key** links a record to a valid record in another table. Other constraints enforce rules such as uniqueness or required values, while triggers demonstrate validation and auditing, views restrict what a user can see, and DCL commands control who may access which table. Together these features support data integrity, security and consistent records.

1.6 Proposed System Overview

The ER model contains ten core entities: Department, Student, Faculty, Course, Enrollment, Attendance, Examination, Marks, Fee and UserAccount. A supporting Student_Audit table is used for the trigger demonstrations and is not one of the ten core entities. The attributes and purpose of each entity are summarised below (PK = primary key, FK = foreign key); the keys are identified in Section 3.2.

Entity	Attributes	Purpose
Department	Department_ID (PK), Department_Name, HOD_Name, Office_Location, Contact_Number	Department details
Student	Student_ID (PK), Student_Name, Date_of_Birth, Phone, Email, Department_ID (FK)	Student personal and contact details
Faculty	Faculty_ID (PK), Faculty_Name, Email, Phone, Designation, Department_ID (FK)	Faculty details and department
Course	Course_ID (PK), Course_Name, Credits, Department_ID (FK), Faculty_ID (FK)	Course details, department and faculty
Enrollment	Enrollment_ID (PK), Student_ID (FK), Course_ID (FK), Semester, Academic_Year, Enrollment_Date, Status	Courses in which students are enrolled
Attendance	Attendance_ID (PK), Enrollment_ID (FK), Attendance_Date, Status, Remarks	Attendance for each enrollment
Examination	Exam_ID (PK), Course_ID (FK), Exam_Name, Exam_Date, Max_Marks, Exam_Type	Examination details for each course
Marks	Mark_ID (PK), Enrollment_ID (FK), Exam_ID (FK), Marks_Obtained, Grade, Remarks	Marks, grades and remarks
Fee	Fee_ID (PK), Student_ID (FK), Amount, Due_Date, Payment_Date, Payment_Status, Payment_Method	Student fee and payment status
UserAccount	Account_ID (PK), Student_ID (FK, UNIQUE), Username (UNIQUE), Password_Hash, Role, Created_At	Student account information

Department is related to students, faculty and courses. Courses are associated with a department and a faculty member. Enrollment links students and courses, allowing a student to enroll in multiple courses and a course to have multiple students. Attendance refers to an enrollment; Examination belongs to a course; and Marks connects an enrollment with an examination. Fee

records refer to students, while UserAccount is linked to a student. Primary and foreign keys define these relationships, as shown in the ER diagram.

1.7 Technologies Used

MySQL is the relational database management system used, with MySQL client-style SQL syntax. The database is named StudentInformationSystem. SQL is used to create and populate tables, retrieve and update records, and demonstrate joins, aggregate queries, subqueries, views, stored procedures, triggers, and data and transaction control. A MySQL client (command-line client or MySQL Workbench) is required to run the examples, and a word-processing application is used to prepare the report and export it to PDF.

1.8 Conclusion

The Student Information System demonstrates how MySQL and relational database concepts can organize student-related academic records. Its connected tables, keys, constraints, views and SQL examples provide a structured approach to storing, retrieving, securing and validating information. The remaining sections present the requirement analysis, ER design, implementation, queries and results in detail.

2. Requirement Analysis

Requirement analysis describes the data, operations and quality conditions needed for the Student Information System. The project is a MySQL relational database demonstration that organizes academic records, connects them through primary and foreign keys, and retrieves information through SQL queries.

2.1 Hardware Requirements

The following are reasonable minimum resources for developing and demonstrating the MySQL project; they are recommendations, not measured specifications of the computers used by the group.

Component	Suggested requirement
Computer	Desktop or laptop
Processor	Dual-core processor or better
Memory	4 GB RAM or more
Storage	At least 1 GB free for MySQL, scripts and documentation
Display / input	Monitor, keyboard and mouse or equivalent

2.2 Software Requirements

Software	Requirement / purpose
RDBMS	MySQL – create and manage the StudentInformationSystem database
SQL execution environment	MySQL command-line client or a compatible interface (such as MySQL Workbench) that supports the demonstrated SQL features
Operating system	Any supported operating system capable of running the chosen MySQL environment
Documentation tool	Word-compatible editor for report preparation and PDF export

2.3 Functional Requirements

Functional requirements state what the database must store and what the documented SQL operations are intended to accomplish.

2.3.1 Department Management

- Maintain Department_ID, Department_Name, HOD_Name, Office_Location and Contact_Number.
- Identify each department using Department_ID and associate its students, faculty and courses with it.
- Retrieve department details and department-wise student counts.

2.3.2 Student Management

- Store Student_ID, Student_Name, Date_of_Birth, Phone, Email and Department_ID.
- Support inserting, selecting, updating and deleting student records through the documented DML operations.

- Retrieve student names together with their department details using joins and subqueries.

2.3.3 Faculty and Course Management

- Store faculty details including Faculty_ID, Faculty_Name, Email, Phone, Designation and Department_ID.
- Store Course_ID, Course_Name, Credits, Department_ID and Faculty_ID for courses.
- Link courses to their department and faculty member, and retrieve courses taught by faculty with a specified designation.

2.3.4 Enrollment Management

- Maintain Enrollment_ID, Student_ID, Course_ID, Semester, Academic_Year, Enrollment_Date and Status.
- Use Enrollment to represent the many-to-many association between students and courses.
- Allow the courses taken by a student and the students of a course to be retrieved through Enrollment.

2.3.5 Attendance Management

- Record Attendance_ID, Enrollment_ID, Attendance_Date, Status and Remarks.
- Associate each attendance entry with a valid enrollment and record statuses such as Present and Absent.
- Retrieve attendance information for students and summarize attendance by status.

2.3.6 Examination and Marks Management

- Maintain examination details using Exam_ID, Course_ID, Exam_Name, Exam_Date, Max_Marks and Exam_Type.
- Store Mark_ID, Enrollment_ID, Exam_ID, Marks_Obtained, Grade and Remarks.
- Link marks to the corresponding enrollment and examination; retrieve student marks, grades and course-wise averages.
- Use validation triggers to reject marks outside the allowed range during insert and update operations.

2.3.7 Fee Management

- Maintain Fee_ID, Student_ID, Amount, Due_Date, Payment_Date, Payment_Status and Payment_Method.
- Link each fee entry to its student and distinguish paid and pending records.
- Support the trigger that blocks deletion of pending fee records and a view that lists pending fees.

2.3.8 User Account Management

- Store Account_ID, Student_ID, Username, Password_Hash, Role and Created_At.
- Associate each user account with a student and maintain unique account identifiers and usernames.
- Treat the sample password-hash values as demonstration data; no login interface is built.

2.3.9 SQL Query and Reporting Operations

- Support SELECT, INSERT, UPDATE and DELETE operations on the database.
- Use ORDER BY, COUNT, SUM, AVG, MIN, MAX, GROUP BY and HAVING to sort and summarize records.

- Use INNER, LEFT, RIGHT and CROSS JOIN, and demonstrate full outer join behaviour through UNION of LEFT and RIGHT JOIN because MySQL has no FULL OUTER JOIN keyword.
- Use nested queries to retrieve students by department, marks and attendance, and courses by faculty designation.

2.3.10 Views, Procedures and Triggers

- Create views for student–department details, department student counts, student marks and pending fees, and update data through an updatable view.
- Support six stored-procedure demonstrations: list students, retrieve one student, list courses with faculty and department, retrieve student marks, retrieve attendance, and count students in a department.
- Support six trigger demonstrations: validate inserted marks, log new students, validate updated marks, audit student updates, prevent deletion of pending fees, and log deleted students.

2.3.11 Data Control and Transaction Control

- Grant and revoke table-level privileges for a database user using GRANT and REVOKE.
- Save or cancel groups of changes using COMMIT, ROLLBACK and SAVEPOINT.

2.4 Non-Functional Requirements

Requirement	Description
Data integrity	Primary keys, foreign keys and declared uniqueness or nullability constraints keep relationships consistent. Marks-validation triggers prevent out-of-range values.
Reliability	SQL operations should give repeatable results for the sample records. Destructive commands such as DROP TABLE and TRUNCATE TABLE are demonstrated on a backup copy.
Performance	The sample database should retrieve student, enrollment, attendance and marks information within a reasonable time for a small academic demonstration.
Security	Access to student and fee data should be restricted in a deployed system. Views and GRANT/REVOKE privileges restrict what a user can see and do; the UserAccount table by itself is not a secure authentication system.
Maintainability and usability	Consistent table names, keys and readable outputs make the database easy to understand and update. Procedures and views centralize frequently used retrieval logic.
Backup and compatibility	Keep backup copies of the SQL scripts, sample records and report. Procedures, triggers and views need a MySQL environment that supports their syntax; transactions need the InnoDB engine.

2.5 Integration Requirements

The ten entities are connected using the relationships shown in the ER diagram. These links allow the database to combine information stored in separate tables without repeating complete student or course details.

Relationship	Key / integration requirement
Department – Student / Faculty / Course	Department_ID is referenced by Student, Faculty and Course (1:N).
Faculty – Course	Course.Faculty_ID references Faculty.Faculty_ID (1:N).
Student – Course	Enrollment connects Student_ID and Course_ID (many-to-many via Enrollment).
Enrollment – Attendance	Attendance.Enrollment_ID references Enrollment.Enrollment_ID (1:N).
Course – Examination	Examination.Course_ID references Course.Course_ID (1:N).
Enrollment / Examination – Marks	Marks.Enrollment_ID and Marks.Exam_ID reference their respective parent tables.
Student – Fee	Fee.Student_ID references Student.Student_ID (1:N).
Student – UserAccount	UserAccount.Student_ID references Student.Student_ID; the UNIQUE Student_ID gives one account per student.

2.6 Project Scope and Feasibility

Technical feasibility. MySQL supports the relational schema and every SQL statement demonstrated in the project. The primary-key and foreign-key design is a practical basis for storing and connecting student information, and views, stored procedures and triggers provide additional database-side logic.

Economic feasibility. The database can be demonstrated on a standard student computer using freely available MySQL tools and sample records, so no specialized hardware is needed.

Operational feasibility. The database is suitable for a classroom demonstration because its operations can be executed and verified with sample student, course, enrollment, attendance, examination, marks and fee records, and it supports meaningful reports through joins, aggregate queries and views.

2.7 Conclusion

The requirement analysis defines the proposed environment, required data operations, quality expectations and table relationships for the Student Information System. These requirements are consistent with the ten-entity ER model, the MySQL implementation and the SQL demonstrations in the remaining sections.

3. ER Model and Database Design

The Entity Relationship (ER) diagram represents the logical structure of the Student Information System database. It shows how the 10 entities are connected using primary keys, foreign keys and relationships. The design uses relational concepts to maintain data integrity and organize student-related information efficiently.

3.1 Entity Relationship Diagram

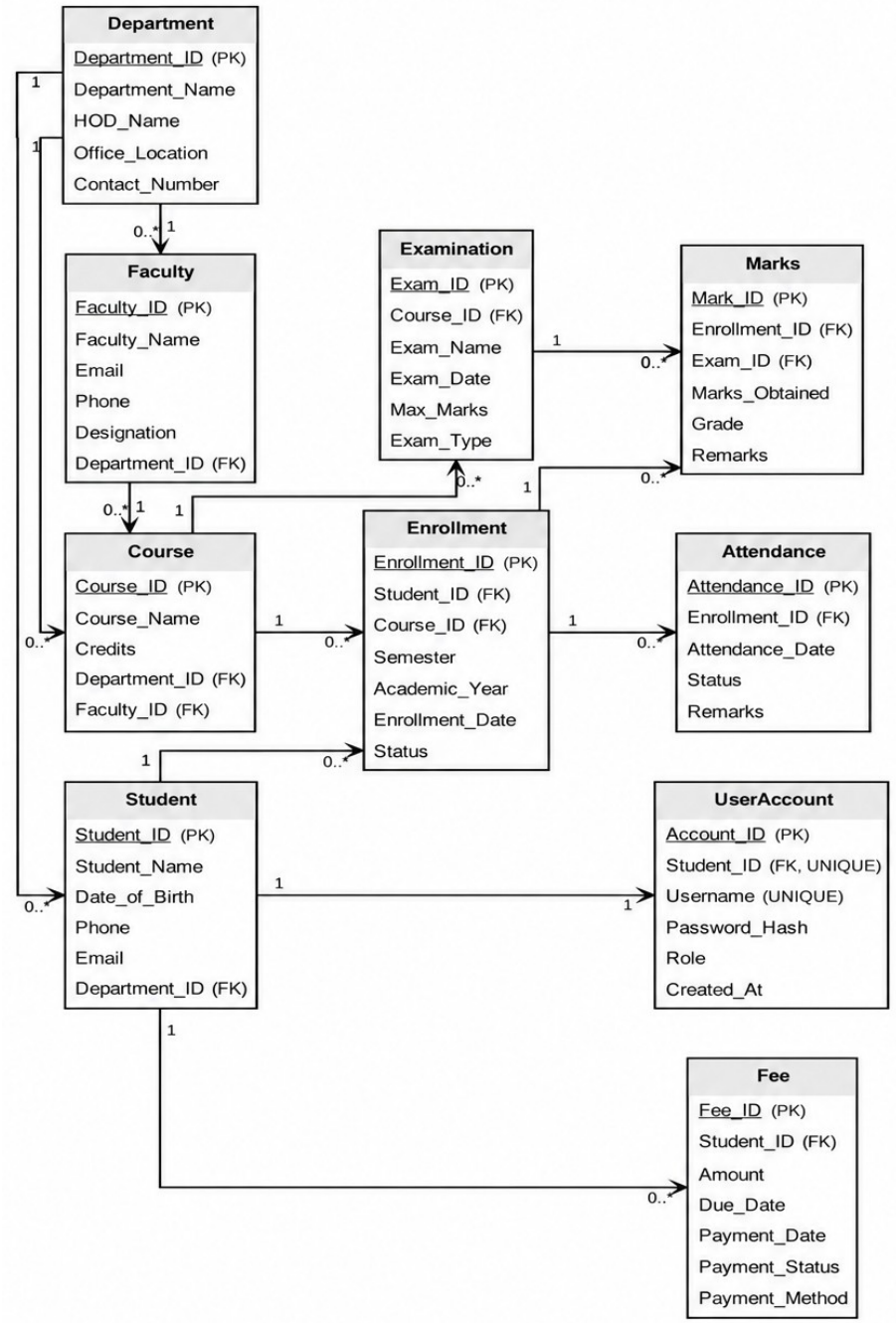


Figure 3.1: ER diagram (class diagram format) of the Student Information System

3.2 Entities and Key Identification

The database consists of 10 entities. Their attributes and purposes were introduced in Section 1; this section identifies the primary keys, candidate keys and foreign keys used in the ER design.

PK (Primary Key) uniquely identifies each record. **CK (Candidate Key)** is an attribute or combination of attributes that can uniquely identify a record. **FK (Foreign Key)** references the primary or unique key of a related table. A **composite key** is formed from two or more attributes.

Entity	Primary key	Foreign key	Candidate key
Department	Department_ID	-	Contact_Number
Student	Student_ID	Department_ID	Phone
Faculty	Faculty_ID	Department_ID	Email, Phone
Course	Course_ID	Department_ID, Faculty_ID	-
Enrollment	Enrollment_ID	Student_ID, Course_ID	-
Attendance	Attendance_ID	Enrollment_ID	-
Examination	Exam_ID	Course_ID	-
Marks	Mark_ID	Enrollment_ID, Exam_ID	-
Fee	Fee_ID	Student_ID	-
UserAccount	Account_ID	Student_ID	Username, Password_Hash

3.3 Relationships and Cardinalities

A **relationship** represents the association between two or more entities. **Cardinality** defines the number of instances of one entity that can be associated with instances of another: One-to-One (1:1), One-to-Many (1:N) and Many-to-Many (M:N).

3.3.1 Direct Relationships

One-to-One:

S.No	Entity relation	Connecting key	Cardinality
1	Student – UserAccount	UserAccount.Student_ID	1 : 1

One-to-Many:

S.No	Entity relation	Connecting key	Cardinality
1	Department – Faculty	Faculty.Department_ID	1 : N
2	Department – Course	Course.Department_ID	1 : N
3	Department – Student	Student.Department_ID	1 : N
4	Faculty – Course	Course.Faculty_ID	1 : N
5	Course – Examination	Examination.Course_ID	1 : N
6	Course – Enrollment	Enrollment.Course_ID	1 : N
7	Student – Enrollment	Enrollment.Student_ID	1 : N
8	Student – Fee	Fee.Student_ID	1 : N
9	Examination – Marks	Marks.Exam_ID	1 : N
10	Enrollment – Marks	Marks.Enrollment_ID	1 : N
11	Enrollment – Attendance	Attendance.Enrollment_ID	1 : N

Explanation of major relationships

1. **Department – Faculty:** one department can have many faculty members; each faculty member belongs to one department.
2. **Department – Course:** one department can offer many courses; each course belongs to one department.
3. **Department – Student:** one department can have many students; each student belongs to one department.
4. **Faculty – Course:** one faculty member can teach many courses; each course is associated with one faculty member.
5. **Course – Examination:** one course can have many examinations; each examination belongs to one course.
6. **Course – Enrollment:** one course can have many enrollment records; each enrollment refers to one course.
7. **Student – Enrollment:** one student can have many enrollment records; each enrollment belongs to one student.
8. **Student – Fee:** one student can have many fee records; each fee record belongs to one student.
9. **Examination – Marks:** one examination can have marks records for many students; each marks record refers to one examination.
10. **Enrollment – Marks:** one enrollment can have many marks records; each marks record is associated with one enrollment.
11. **Enrollment – Attendance:** one enrollment can have many attendance records; each attendance record is associated with one enrollment.
12. **Student – UserAccount:** a student can have one user account, and each user account belongs to one student.

3.3.2 Indirect Many-to-Many Relationships

A many-to-many relationship occurs when multiple records in one table are associated with multiple records in another. Such relationships are implemented using junction tables.

No.	Related entities	Junction table
1	Student – Course	Enrollment
2	Enrollment – Examination	Marks

Student – Course: one student can enroll in many courses and one course can have many students; the Enrollment table resolves this relationship. **Enrollment – Examination:** one enrollment can be associated with many examinations and one examination with many enrollments; the Marks table connects them and stores the marks obtained.

4. Relational Model and Database Implementation

The Student Information System is implemented using MySQL. The implementation demonstrates database creation, table creation (with attributes, data types, primary keys, foreign keys and constraints), insertion of sample student-related records, and verification of the stored records using SQL commands. All outputs are shown as they appear in the MySQL command-line client.

4.1 Database Creation

The first step is to create and select the database for the Student Information System.

SQL Command and Output:

```
mysql> CREATE DATABASE StudentInformationSystem;
Query OK, 1 row affected (0.002 sec)

mysql> USE StudentInformationSystem;
Database changed

mysql> SHOW DATABASES;
+--------------------------+
| Database                 |
+--------------------------+
| information_schema       |
| mysql                    |
| performance_schema       |
| StudentInformationSystem |
| sys                      |
+--------------------------+
5 rows in set (0.002 sec)
```

Note: If the database has already been created and selected, the commands do not need to be executed again.

4.2 Table Creation, Data Insertion and Verification

The database contains 10 tables. Each table is created using a CREATE TABLE statement, populated with sample records using INSERT INTO, and verified using SELECT. Tables are created in an order that respects the foreign keys: parent tables first, child tables afterwards.

4.2.1 Department Table

Purpose: Stores department details, including the department name, head of department, office location and contact number.

Table Creation (DDL):

```
mysql> CREATE TABLE Department (
    ->     Department_ID INT PRIMARY KEY,
    ->     Department_Name VARCHAR(100) NOT NULL,
    ->     HOD_Name VARCHAR(100),
    ->     Office_Location VARCHAR(100),
    ->     Contact_Number VARCHAR(15)
    -> );
Query OK, 0 rows affected (0.035 sec)
```

Data Insertion (DML):

```
mysql> INSERT INTO Department
    -> (Department_ID, Department_Name, HOD_Name, Office_Location, Contact_Number)
    -> VALUES
    -> (1, 'Computer Science', 'Dr. Kumar', 'Block A', '9876500001'),
    -> (2, 'Information Technology', 'Dr. Sharma', 'Block B', '9876500002'),
    -> (3, 'Electronics', 'Dr. Reddy', 'Block C', '9876500003'),
    -> (4, 'Mechanical Engineering', 'Dr. Rao', 'Block D', '9876500004'),
    -> (5, 'Civil Engineering', 'Dr. Singh', 'Block E', '9876500005');
Query OK, 5 rows affected (0.006 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Verification Query and Output:

```
mysql> SELECT * FROM Department;
+---------------+-----------------------+------------+-----------------+----------------+
| Department_ID | Department_Name       | HOD_Name   | Office_Location | Contact_Number |
+---------------+-----------------------+------------+-----------------+----------------+
|             1 | Computer Science      | Dr. Kumar  | Block A         | 9876500001     |
|             2 | Information Technology | Dr. Sharma | Block B         | 9876500002     |
|             3 | Electronics           | Dr. Reddy  | Block C         | 9876500003     |
|             4 | Mechanical Engineering | Dr. Rao    | Block D         | 9876500004     |
|             5 | Civil Engineering     | Dr. Singh  | Block E         | 9876500005     |
+---------------+-----------------------+------------+-----------------+----------------+
5 rows in set (0.000 sec)
```

4.2.2 Student Table

Purpose: Stores student personal and contact details and links each student to a department.

Table Creation (DDL):

```
mysql> CREATE TABLE Student (
    ->     Student_ID INT PRIMARY KEY,
    ->     Student_Name VARCHAR(100) NOT NULL,
    ->     Date_of_Birth DATE,
    ->     Phone VARCHAR(15),
    ->     Email VARCHAR(100),
    ->     Department_ID INT,
    ->     FOREIGN KEY (Department_ID)
    ->         REFERENCES Department(Department_ID)
    -> );
Query OK, 0 rows affected (0.035 sec)
```

Data Insertion (DML):

```
mysql> INSERT INTO Student
    -> (Student_ID, Student_Name, Date_of_Birth, Phone, Email, Department_ID)
    -> VALUES
    -> (101, 'Aarav Kumar', '2005-03-12', '9876500101', 'aarav@example.com', 1),
    -> (102, 'Diya Sharma', '2005-07-24', '9876500102', 'diya@example.com', 1),
    -> (103, 'Rahul Reddy', '2004-11-08', '9876500103', 'rahul@example.com', 2),
    -> (104, 'Ananya Rao', '2005-01-16', '9876500104', 'ananya@example.com', 3),
    -> (105, 'Kiran Singh', '2005-09-30', '9876500105', 'kiran@example.com', 2);
Query OK, 5 rows affected (0.006 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Verification Query and Output:

```
mysql> SELECT * FROM Student;
+------------+--------------+---------------+------------+--------------------+---------------+
| Student_ID | Student_Name | Date_of_Birth | Phone      | Email              | Department_ID |
+------------+--------------+---------------+------------+--------------------+---------------+
|        101 | Aarav Kumar  | 2005-03-12    | 9876500101 | aarav@example.com  |             1 |
|        102 | Diya Sharma  | 2005-07-24    | 9876500102 | diya@example.com   |             1 |
|        103 | Rahul Reddy  | 2004-11-08    | 9876500103 | rahul@example.com  |             2 |
|        104 | Ananya Rao   | 2005-01-16    | 9876500104 | ananya@example.com |             3 |
|        105 | Kiran Singh  | 2005-09-30    | 9876500105 | kiran@example.com  |             2 |
+------------+--------------+---------------+------------+--------------------+---------------+
5 rows in set (0.003 sec)
```

4.2.3 Faculty Table

Purpose: Stores faculty member details, including contact information, designation, and the department to which each faculty member belongs.

Table Creation (DDL):

```
mysql> CREATE TABLE Faculty (
    ->     Faculty_ID INT PRIMARY KEY,
    ->     Faculty_Name VARCHAR(100) NOT NULL,
    ->     Email VARCHAR(100),
    ->     Phone VARCHAR(15),
    ->     Designation VARCHAR(50),
    ->     Department_ID INT,
    ->     FOREIGN KEY (Department_ID)
    ->         REFERENCES Department(Department_ID)
    -> );
Query OK, 0 rows affected (0.035 sec)
```

Data Insertion (DML):

```
mysql> INSERT INTO Faculty
    -> (Faculty_ID, Faculty_Name, Email, Phone, Designation, Department_ID)
    -> VALUES
    -> (201, 'Dr. Meera', 'meera@example.com', '9876500201', 'Professor', 1),
    -> (202, 'Mr. Arjun', 'arjun@example.com', '9876500202', 'Assistant Professor', 2),
    -> (203, 'Dr. Kavita', 'kavita@example.com', '9876500203', 'Professor', 3),
    -> (204, 'Mr. Suresh', 'suresh@example.com', '9876500204', 'Lecturer', 4),
    -> (205, 'Dr. Priya', 'priya@example.com', '9876500205', 'Assistant Professor', 5);
Query OK, 5 rows affected (0.006 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Verification Query and Output:

```
mysql> SELECT * FROM Faculty;
+------------+--------------+--------------------+------------+--------------------+---------------+
| Faculty_ID | Faculty_Name | Email              | Phone      | Designation        | Department_ID |
+------------+--------------+--------------------+------------+--------------------+---------------+
|        201 | Dr. Meera    | meera@example.com  | 9876500201 | Professor          |             1 |
|        202 | Mr. Arjun    | arjun@example.com  | 9876500202 | Assistant Professor |             2 |
|        203 | Dr. Kavita   | kavita@example.com | 9876500203 | Professor          |             3 |
|        204 | Mr. Suresh   | suresh@example.com | 9876500204 | Lecturer           |             4 |
|        205 | Dr. Priya    | priya@example.com  | 9876500205 | Assistant Professor |             5 |
+------------+--------------+--------------------+------------+--------------------+---------------+
5 rows in set (0.001 sec)
```

4.2.4 Course Table

Purpose: Stores course details such as course name and credits, along with the department and faculty responsible for the course.

Table Creation (DDL):

```
mysql> CREATE TABLE Course (
    ->     Course_ID INT PRIMARY KEY,
    ->     Course_Name VARCHAR(100) NOT NULL,
    ->     Credits INT,
    ->     Department_ID INT,
    ->     Faculty_ID INT,
    ->     FOREIGN KEY (Department_ID)
    ->         REFERENCES Department(Department_ID),
    ->     FOREIGN KEY (Faculty_ID)
    ->         REFERENCES Faculty(Faculty_ID)
    -> );
Query OK, 0 rows affected (0.035 sec)
```

Data Insertion (DML):

```
mysql> INSERT INTO Course
    -> (Course_ID, Course_Name, Credits, Department_ID, Faculty_ID)
    -> VALUES
    -> (301, 'DBMS', 4, 1, 201),
    -> (302, 'Computer Networks', 3, 2, 202),
    -> (303, 'Digital Electronics', 4, 3, 203),
    -> (304, 'Engineering Mechanics', 3, 4, 204),
    -> (305, 'Structural Engineering', 4, 5, 205);
Query OK, 5 rows affected (0.006 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Verification Query and Output:

```
mysql> SELECT * FROM Course;
+-----------+-----------------------+---------+---------------+------------+
| Course_ID | Course_Name           | Credits | Department_ID | Faculty_ID |
+-----------+-----------------------+---------+---------------+------------+
|       301 | DBMS                  |       4 |             1 |        201 |
|       302 | Computer Networks     |       3 |             2 |        202 |
|       303 | Digital Electronics   |       4 |             3 |        203 |
|       304 | Engineering Mechanics |       3 |             4 |        204 |
|       305 | Structural Engineering |       4 |             5 |        205 |
+-----------+-----------------------+---------+---------------+------------+
5 rows in set (0.004 sec)
```

4.2.5 Enrollment Table

Purpose: Records the courses in which students are enrolled, including semester, academic year, enrollment date and enrollment status.

Table Creation (DDL):

```
mysql> CREATE TABLE Enrollment (
    ->     Enrollment_ID INT PRIMARY KEY,
    ->     Student_ID INT NOT NULL,
    ->     Course_ID INT NOT NULL,
    ->     Semester INT,
    ->     Academic_Year VARCHAR(9),
    ->     Enrollment_Date DATE,
    ->     Status VARCHAR(20),
    ->     FOREIGN KEY (Student_ID)
    ->         REFERENCES Student(Student_ID),
    ->     FOREIGN KEY (Course_ID)
    ->         REFERENCES Course(Course_ID)
    -> );
Query OK, 0 rows affected (0.035 sec)
```

Data Insertion (DML):

```
mysql> INSERT INTO Enrollment
    -> (Enrollment_ID, Student_ID, Course_ID, Semester, Academic_Year, Enrollment_Date, Status)
    -> VALUES
    -> (401, 101, 301, 3, '2026-2027', '2026-07-01', 'Active'),
    -> (402, 102, 301, 3, '2026-2027', '2026-07-02', 'Active'),
    -> (403, 103, 302, 3, '2026-2027', '2026-07-03', 'Active'),
    -> (404, 104, 303, 3, '2026-2027', '2026-07-04', 'Active'),
    -> (405, 105, 302, 3, '2026-2027', '2026-07-05', 'Active');
Query OK, 5 rows affected (0.006 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Verification Query and Output:

```
mysql> SELECT * FROM Enrollment;
+---------------+------------+-----------+----------+---------------+-----------------+--------+
| Enrollment_ID | Student_ID | Course_ID | Semester | Academic_Year | Enrollment_Date | Status |
+---------------+------------+-----------+----------+---------------+-----------------+--------+
|           401 |        101 |       301 |        3 | 2026-2027     | 2026-07-01      | Active |
|           402 |        102 |       301 |        3 | 2026-2027     | 2026-07-02      | Active |
|           403 |        103 |       302 |        3 | 2026-2027     | 2026-07-03      | Active |
|           404 |        104 |       303 |        3 | 2026-2027     | 2026-07-04      | Active |
|           405 |        105 |       302 |        3 | 2026-2027     | 2026-07-05      | Active |
+---------------+------------+-----------+----------+---------------+-----------------+--------+
5 rows in set (0.002 sec)
```

4.2.6 Attendance Table

Purpose: Stores attendance records for each enrollment, including the attendance date, present/absent status and remarks.

Table Creation (DDL):

```
mysql> CREATE TABLE Attendance (
    ->     Attendance_ID INT PRIMARY KEY,
    ->     Enrollment_ID INT NOT NULL,
    ->     Attendance_Date DATE NOT NULL,
    ->     Status VARCHAR(10) NOT NULL,
    ->     Remarks VARCHAR(200),
    ->     FOREIGN KEY (Enrollment_ID)
    ->         REFERENCES Enrollment(Enrollment_ID)
    -> );
Query OK, 0 rows affected (0.035 sec)
```

Data Insertion (DML):

```
mysql> INSERT INTO Attendance
    -> (Attendance_ID, Enrollment_ID, Attendance_Date, Status, Remarks)
    -> VALUES
    -> (701, 401, '2026-07-10', 'Present', 'Attended class'),
    -> (702, 402, '2026-07-10', 'Absent', 'Absent from class'),
    -> (703, 403, '2026-07-10', 'Present', 'Attended class'),
    -> (704, 404, '2026-07-10', 'Present', 'Attended class'),
    -> (705, 405, '2026-07-10', 'Absent', 'Leave not recorded');
Query OK, 5 rows affected (0.006 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Verification Query and Output:

```
mysql> SELECT * FROM Attendance;
+---------------+---------------+-----------------+---------+--------------------+
| Attendance_ID | Enrollment_ID | Attendance_Date | Status  | Remarks            |
+---------------+---------------+-----------------+---------+--------------------+
|           701 |           401 | 2026-07-10      | Present | Attended class     |
|           702 |           402 | 2026-07-10      | Absent  | Absent from class  |
|           703 |           403 | 2026-07-10      | Present | Attended class     |
|           704 |           404 | 2026-07-10      | Present | Attended class     |
|           705 |           405 | 2026-07-10      | Absent  | Leave not recorded |
+---------------+---------------+-----------------+---------+--------------------+
5 rows in set (0.001 sec)
```

4.2.7 Examination Table

Purpose: Stores examination details for each course, including examination name, date, maximum marks and examination type.

Table Creation (DDL):

```
mysql> CREATE TABLE Examination (
    ->     Exam_ID INT PRIMARY KEY,
    ->     Course_ID INT NOT NULL,
    ->     Exam_Name VARCHAR(50) NOT NULL,
    ->     Exam_Date DATE,
    ->     Max_Marks DECIMAL(5,2),
    ->     Exam_Type VARCHAR(30),
    ->     FOREIGN KEY (Course_ID)
    ->         REFERENCES Course(Course_ID)
    -> );
Query OK, 0 rows affected (0.035 sec)
```

Data Insertion (DML):

```
mysql> INSERT INTO Examination
    -> (Exam_ID, Course_ID, Exam_Name, Exam_Date, Max_Marks, Exam_Type)
    -> VALUES
    -> (501, 301, 'DBMS Internal 1', '2026-08-10', 100, 'Internal'),
    -> (502, 301, 'DBMS Internal 2', '2026-09-10', 100, 'Internal'),
    -> (503, 302, 'Networks Internal 1', '2026-08-12', 100, 'Internal'),
    -> (504, 303, 'Electronics Internal 1', '2026-08-15', 100, 'Internal'),
    -> (505, 302, 'Networks Internal 2', '2026-09-12', 100, 'Internal');
Query OK, 5 rows affected (0.006 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Verification Query and Output:

```
mysql> SELECT * FROM Examination;
+---------+-----------+-----------------------+------------+-----------+-----------+
| Exam_ID | Course_ID | Exam_Name             | Exam_Date  | Max_Marks | Exam_Type |
+---------+-----------+-----------------------+------------+-----------+-----------+
|     501 |       301 | DBMS Internal 1       | 2026-08-10 |    100.00 | Internal  |
|     502 |       301 | DBMS Internal 2       | 2026-09-10 |    100.00 | Internal  |
|     503 |       302 | Networks Internal 1   | 2026-08-12 |    100.00 | Internal  |
|     504 |       303 | Electronics Internal 1 | 2026-08-15 |    100.00 | Internal  |
|     505 |       302 | Networks Internal 2   | 2026-09-12 |    100.00 | Internal  |
+---------+-----------+-----------------------+------------+-----------+-----------+
5 rows in set (0.002 sec)
```

4.2.8 Marks Table

Purpose: Stores marks obtained by students in examinations, along with grades and performance remarks.

Table Creation (DDL):

```
mysql> CREATE TABLE Marks (
    ->     Mark_ID INT PRIMARY KEY,
    ->     Enrollment_ID INT NOT NULL,
    ->     Exam_ID INT NOT NULL,
    ->     Marks_Obtained DECIMAL(5,2),
    ->     Grade VARCHAR(5),
    ->     Remarks VARCHAR(100),
    ->     FOREIGN KEY (Enrollment_ID)
    ->         REFERENCES Enrollment(Enrollment_ID),
    ->     FOREIGN KEY (Exam_ID)
    ->         REFERENCES Examination(Exam_ID)
    -> );
Query OK, 0 rows affected (0.035 sec)
```

Data Insertion (DML):

```
mysql> INSERT INTO Marks
    -> (Mark_ID, Enrollment_ID, Exam_ID, Marks_Obtained, Grade, Remarks)
    -> VALUES
    -> (601, 401, 501, 85, 'A', 'Good performance'),
    -> (602, 402, 502, 90, 'A+', 'Excellent performance'),
    -> (603, 403, 503, 78, 'B+', 'Good performance'),
    -> (604, 404, 504, 88, 'A', 'Very good performance'),
    -> (605, 405, 505, 72, 'B', 'Satisfactory performance');
Query OK, 5 rows affected (0.006 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Verification Query and Output:

```
mysql> SELECT * FROM Marks;
+---------+---------------+---------+---------------+-------+--------------------------+
| Mark_ID | Enrollment_ID | Exam_ID | Marks_Obtained | Grade | Remarks                  |
+---------+---------------+---------+---------------+-------+--------------------------+
|     601 |           401 |     501 |          85.00 | A     | Good performance         |
|     602 |           402 |     502 |          90.00 | A+    | Excellent performance    |
|     603 |           403 |     503 |          78.00 | B+    | Good performance         |
|     604 |           404 |     504 |          88.00 | A     | Very good performance    |
|     605 |           405 |     505 |          72.00 | B     | Satisfactory performance |
+---------+---------------+---------+---------------+-------+--------------------------+
5 rows in set (0.000 sec)
```

4.2.9 Fee Table

Purpose: Maintains student fee details, including amount due, due date, payment date, payment status and payment method.

Table Creation (DDL):

```
mysql> CREATE TABLE Fee (
    ->     Fee_ID INT PRIMARY KEY,
    ->     Student_ID INT NOT NULL,
    ->     Amount DECIMAL(10,2) NOT NULL,
    ->     Due_Date DATE,
    ->     Payment_Date DATE,
    ->     Payment_Status VARCHAR(20),
    ->     Payment_Method VARCHAR(30),
    ->     FOREIGN KEY (Student_ID)
    ->         REFERENCES Student(Student_ID)
    -> );
Query OK, 0 rows affected (0.035 sec)
```

Data Insertion (DML):

```
mysql> INSERT INTO Fee
    -> (Fee_ID, Student_ID, Amount, Due_Date, Payment_Date, Payment_Status, Payment_Method)
    -> VALUES
    -> (801, 101, 45000.00, '2026-07-15', '2026-07-10', 'Paid', 'UPI'),
    -> (802, 102, 45000.00, '2026-07-15', NULL, 'Pending', NULL),
    -> (803, 103, 42000.00, '2026-07-15', '2026-07-12', 'Paid', 'Bank Transfer'),
    -> (804, 104, 48000.00, '2026-07-15', NULL, 'Pending', NULL),
    -> (805, 105, 42000.00, '2026-07-15', '2026-07-11', 'Paid', 'Cash');
Query OK, 5 rows affected (0.006 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Verification Query and Output:

```
mysql> SELECT * FROM Fee;
+--------+------------+----------+------------+--------------+----------------+----------------+
| Fee_ID | Student_ID | Amount   | Due_Date   | Payment_Date | Payment_Status | Payment_Method |
+--------+------------+----------+------------+--------------+----------------+----------------+
|    801 |        101 | 45000.00 | 2026-07-15 | 2026-07-10   | Paid           | UPI            |
|    802 |        102 | 45000.00 | 2026-07-15 | NULL         | Pending        | NULL           |
|    803 |        103 | 42000.00 | 2026-07-15 | 2026-07-12   | Paid           | Bank Transfer  |
|    804 |        104 | 48000.00 | 2026-07-15 | NULL         | Pending        | NULL           |
|    805 |        105 | 42000.00 | 2026-07-15 | 2026-07-11   | Paid           | Cash           |
+--------+------------+----------+------------+--------------+----------------+----------------+
5 rows in set (0.003 sec)
```

4.2.10 UserAccount Table

Purpose: Stores student account information, including username, password-hash placeholder, role and account creation date.

Table Creation (DDL):

```
mysql> CREATE TABLE UserAccount (
    ->     Account_ID INT PRIMARY KEY,
    ->     Student_ID INT NOT NULL UNIQUE,
    ->     Username VARCHAR(50) NOT NULL UNIQUE,
    ->     Password_Hash VARCHAR(255) NOT NULL,
    ->     Role VARCHAR(20),
    ->     Created_At DATE,
    ->     FOREIGN KEY (Student_ID)
    ->         REFERENCES Student(Student_ID)
    -> );
Query OK, 0 rows affected (0.036 sec)
```

Data Insertion (DML):

```
mysql> INSERT INTO UserAccount
    -> (Account_ID, Student_ID, Username, Password_Hash, Role, Created_At)
    -> VALUES
    -> (901, 101, 'aarav101', 'sample_hash_101', 'Student', '2026-07-01'),
    -> (902, 102, 'diya102', 'sample_hash_102', 'Student', '2026-07-02'),
    -> (903, 103, 'rahul103', 'sample_hash_103', 'Student', '2026-07-03'),
    -> (904, 104, 'ananya104', 'sample_hash_104', 'Student', '2026-07-04'),
    -> (905, 105, 'kiran105', 'sample_hash_105', 'Student', '2026-07-05');
Query OK, 5 rows affected (0.007 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Verification Query and Output:

```
mysql> SELECT * FROM UserAccount;
+------------+------------+-----------+-----------------+---------+------------+
| Account_ID | Student_ID | Username  | Password_Hash   | Role    | Created_At |
+------------+------------+-----------+-----------------+---------+------------+
|        901 |        101 | aarav101  | sample_hash_101 | Student | 2026-07-01 |
|        902 |        102 | diya102   | sample_hash_102 | Student | 2026-07-02 |
|        903 |        103 | rahul103  | sample_hash_103 | Student | 2026-07-03 |
|        904 |        104 | ananya104 | sample_hash_104 | Student | 2026-07-04 |
|        905 |        105 | kiran105  | sample_hash_105 | Student | 2026-07-05 |
+------------+------------+-----------+-----------------+---------+------------+
5 rows in set (0.001 sec)
```

5. SQL Fundamentals and Queries

This section presents the SQL queries and operations implemented on the Student Information System database, arranged from basic to advanced: DDL commands, DML commands, sorting and aggregate functions, joins, aggregate queries with joins (GROUP BY and HAVING), nested queries, stored procedures and triggers. For every query the description, the SQL code (as typed in the MySQL client) and the corresponding output are given.

5.1 Data Definition Language (DDL) Commands

These commands use tables that already exist in the StudentInformationSystem database. ALTER TABLE changes the structure of a table, whereas DROP TABLE and TRUNCATE TABLE are destructive commands that remove a table or all of its rows. Because they cannot be undone, the commands in this section were executed on a backup copy of the database, so that the sample data used by the remaining queries is not affected.

5.1.1 ALTER TABLE Command

Aim / Purpose: To add an Address column to the existing Student table.

SQL Command and Output:

```
mysql> ALTER TABLE Student
    -> ADD COLUMN Address VARCHAR(150);
Query OK, 0 rows affected (0.005 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> DESCRIBE Student;
+--------------+-------------+------+-----+---------+-------+
| Field        | Type        | Null | Key | Default | Extra |
+--------------+-------------+------+-----+---------+-------+
| Student_ID   | int         | NO   | PRI | NULL    |       |
| Student_Name | varchar(100) | NO   |     | NULL    |       |
| Date_of_Birth | date       | YES  |     | NULL    |       |
| Phone        | varchar(15)  | YES  |     | NULL    |       |
| Email        | varchar(100) | YES  |     | NULL    |       |
| Department_ID | int        | YES  | MUL | NULL    |       |
| Address      | varchar(150) | YES  |     | NULL    |       |
+--------------+-------------+------+-----+---------+-------+
7 rows in set (0.004 sec)
```

Explanation: ALTER TABLE changes the structure of an existing table without deleting its current student records. If Address already exists, do not run this command again. The DESCRIBE output shows the new Address column.

5.1.2 DROP TABLE Command

Aim / Purpose: To demonstrate removal of an existing table from the database.

SQL Command and Output:

```
mysql> DROP TABLE UserAccount;
Query OK, 0 rows affected (0.003 sec)
```

Explanation: DROP TABLE permanently removes the table structure and its records.

5.1.3 TRUNCATE TABLE Command

Aim / Purpose: To demonstrate removing all records from the existing Attendance table while retaining its structure.

SQL Command and Output:

```
mysql> TRUNCATE TABLE Attendance;
Query OK, 0 rows affected (0.003 sec)

mysql> SELECT * FROM Attendance;
Empty set (0.000 sec)
```

Explanation: TRUNCATE removes all rows from Attendance; it does not remove the table itself, and the verification query returns an empty set.

5.2 Data Manipulation Language (DML) Commands

Aim: To perform data manipulation operations (SELECT, INSERT, UPDATE and DELETE) on the existing Student Information System database and verify the results using MySQL.

5.2.1 SELECT Command

Aim / Purpose: To display all student records stored in the Student table. SELECT retrieves records.

SQL Command and Output:

```
mysql> SELECT * FROM Student;
+------------+--------------+---------------+------------+--------------------+---------------+
| Student_ID | Student_Name | Date_of_Birth | Phone      | Email              | Department_ID |
+------------+--------------+---------------+------------+--------------------+---------------+
|        101 | Aarav Kumar  | 2005-03-12    | 9876500101 | aarav@example.com  |             1 |
|        102 | Diya Sharma  | 2005-07-24    | 9876500102 | diya@example.com   |             1 |
|        103 | Rahul Reddy  | 2004-11-08    | 9876500103 | rahul@example.com  |             2 |
|        104 | Ananya Rao   | 2005-01-16    | 9876500104 | ananya@example.com |             3 |
|        105 | Kiran Singh  | 2005-09-30    | 9876500105 | kiran@example.com  |             2 |
+------------+--------------+---------------+------------+--------------------+---------------+
5 rows in set (0.002 sec)
```

5.2.2 INSERT Command

Aim / Purpose: To insert one new sample student record into the existing Student table. INSERT adds a new row to a table.

SQL Command and Output:

```
mysql> INSERT INTO Student
    -> (Student_ID, Student_Name, Date_of_Birth, Phone, Email, Department_ID)
    -> VALUES
    -> (106, 'Sneha Patel', '2005-05-14', '9876500106', 'sneha@example.com', 1);
Query OK, 1 row affected (0.001 sec)

mysql> SELECT * FROM Student;
+------------+--------------+---------------+------------+--------------------+---------------+
| Student_ID | Student_Name | Date_of_Birth | Phone      | Email              | Department_ID |
+------------+--------------+---------------+------------+--------------------+---------------+
|        101 | Aarav Kumar  | 2005-03-12    | 9876500101 | aarav@example.com  |             1 |
|        102 | Diya Sharma  | 2005-07-24    | 9876500102 | diya@example.com   |             1 |
|        103 | Rahul Reddy  | 2004-11-08    | 9876500103 | rahul@example.com  |             2 |
|        104 | Ananya Rao   | 2005-01-16    | 9876500104 | ananya@example.com |             3 |
|        105 | Kiran Singh  | 2005-09-30    | 9876500105 | kiran@example.com  |             2 |
|        106 | Sneha Patel  | 2005-05-14    | 9876500106 | sneha@example.com  |             1 |
+------------+--------------+---------------+------------+--------------------+---------------+
6 rows in set (0.001 sec)
```

5.2.3 UPDATE Command

Aim / Purpose: To update the phone number of an existing student record. The WHERE clause ensures only Student_ID 101 is changed.

SQL Command and Output:

```
mysql> UPDATE Student
    -> SET Phone = '9876500199'
    -> WHERE Student_ID = 101;
Query OK, 1 row affected (0.002 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT Student_ID, Student_Name, Phone
    -> FROM Student
    -> WHERE Student_ID = 101;
+------------+--------------+------------+
| Student_ID | Student_Name | Phone      |
+------------+--------------+------------+
|        101 | Aarav Kumar  | 9876500199 |
+------------+--------------+------------+
1 row in set (0.002 sec)
```

5.2.4 DELETE Command

Aim / Purpose: To remove the temporary student record inserted in 5.2.2 from the Student table. DELETE removes the row matching the WHERE condition; run 5.2.2 first, because this example removes only the temporary student 106.

SQL Command and Output:

```
mysql> DELETE FROM Student
    -> WHERE Student_ID = 106;
Query OK, 1 row affected (0.001 sec)

mysql> SELECT * FROM Student WHERE Student_ID = 106;
Empty set (0.000 sec)
```

5.3 Sorting and Aggregate Functions (ORDER BY and GROUP BY)

This section demonstrates how SQL can summarize and organize student-related information using aggregate functions, ORDER BY and GROUP BY on a single table. The queries use the table and column names defined in Section 4.

5.3.1 Concept Overview

Concept	Purpose	Example use
COUNT()	Counts rows or non-NULL values.	Count student records.
SUM()	Adds numeric values.	Calculate the total of recorded marks.
AVG()	Calculates the average.	Find average examination marks.
MIN()	Finds the smallest value.	Find the lowest recorded mark.
MAX()	Finds the largest value.	Find the highest recorded mark.
ORDER BY	Sorts query results using ASC or DESC.	Sort students alphabetically or marks from high to low.
GROUP BY	Groups rows with the same value.	Count students per department.
HAVING	Filters groups after grouping.	Show departments with more than one student.

Important difference: WHERE filters individual rows before grouping. HAVING filters groups after GROUP BY and is commonly used with aggregate conditions such as `COUNT(*) > 1`.

5.3.2 Count the Number of Students

Purpose: Count all records in the Student table. COUNT(*) counts every row in Student.

SQL Command and Output:

```
mysql> USE StudentInformationSystem;
Database changed

mysql> SELECT COUNT(*) AS Total_Students
    -> FROM Student;
+----------------+
| Total_Students |
+----------------+
|              5 |
+----------------+
1 row in set (0.000 sec)
```

Interpretation: The sample data contains five student records.

5.3.3 Aggregate Summary of Marks

Purpose: Calculate the count, total, average, minimum and maximum of recorded marks. This query uses all five aggregate functions on Marks_Obtained; ROUND() displays the average to two decimal places.

SQL Command and Output:

```
mysql> SELECT
    ->     COUNT(Marks_Obtained) AS Marks_Count,
    ->     SUM(Marks_Obtained) AS Total_Marks,
    ->     ROUND(AVG(Marks_Obtained), 2) AS Average_Marks,
    ->     MIN(Marks_Obtained) AS Minimum_Marks,
    ->     MAX(Marks_Obtained) AS Maximum_Marks
    -> FROM Marks;
+-------------+-------------+---------------+---------------+---------------+
| Marks_Count | Total_Marks | Average_Marks | Minimum_Marks | Maximum_Marks |
+-------------+-------------+---------------+---------------+---------------+
|           5 |      413.00 |         82.60 |         72.00 |         90.00 |
+-------------+-------------+---------------+---------------+---------------+
1 row in set (0.000 sec)
```

Interpretation: Five mark records total 413; the average is 82.60, the lowest mark is 72 and the highest is 90.

5.3.4 Display Students in Alphabetical Order

Purpose: List students from A to Z. ASC sorts the names in ascending order and is the default sort direction.

SQL Command and Output:

```
mysql> SELECT Student_ID, Student_Name, Department_ID
    -> FROM Student
    -> ORDER BY Student_Name ASC;
+------------+--------------+---------------+
| Student_ID | Student_Name | Department_ID |
+------------+--------------+---------------+
|        101 | Aarav Kumar  |             1 |
|        104 | Ananya Rao   |             3 |
|        102 | Diya Sharma  |             1 |
|        105 | Kiran Singh  |             2 |
|        103 | Rahul Reddy  |             2 |
+------------+--------------+---------------+
5 rows in set (0.000 sec)
```

5.3.5 Count Attendance by Status

Purpose: Count attendance records for each status. GROUP BY creates groups for Present and Absent, and COUNT(*) counts the records in each group.

SQL Command and Output:

```
mysql> SELECT Status, COUNT(*) AS Attendance_Count
    -> FROM Attendance
    -> GROUP BY Status
    -> ORDER BY Attendance_Count DESC;
+---------+------------------+
| Status  | Attendance_Count |
+---------+------------------+
| Present |                3 |
| Absent  |                2 |
+---------+------------------+
2 rows in set (0.000 sec)
```

Interpretation: The sample has three Present attendance records and two Absent records.

5.4 Joins and Set Operations

A join combines rows from two or more tables using a related column. In the Student Information System the Department_ID column connects the Student and Department tables, so the following queries use these two tables to demonstrate each type of join.

5.4.1 INNER JOIN

Aim / Purpose: To display the details of students along with their corresponding department names. INNER JOIN returns only the rows that have matching values in both tables.

SQL Query and Output:

```
mysql> SELECT Student.Student_ID,
    ->        Student.Student_Name,
    ->        Department.Department_ID,
    ->        Department.Department_Name
    -> FROM Student
    -> INNER JOIN Department
    -> ON Student.Department_ID = Department.Department_ID;
+------------+--------------+---------------+-----------------------+
| Student_ID | Student_Name | Department_ID | Department_Name       |
+------------+--------------+---------------+-----------------------+
|        101 | Aarav Kumar  |             1 | Computer Science      |
|        102 | Diya Sharma  |             1 | Computer Science      |
|        103 | Rahul Reddy  |             2 | Information Technology |
|        104 | Ananya Rao   |             3 | Electronics           |
|        105 | Kiran Singh  |             2 | Information Technology |
+------------+--------------+---------------+-----------------------+
5 rows in set (0.003 sec)
```

5.4.2 RIGHT JOIN

Aim / Purpose: To display all departments along with the students belonging to each department. RIGHT JOIN returns all rows from the right table (Department) and the matching rows from Student; if a department has no students, the student columns contain NULL.

SQL Query and Output:

```
mysql> SELECT Student.Student_ID,
    ->        Student.Student_Name,
    ->        Department.Department_ID,
    ->        Department.Department_Name
    -> FROM Student
    -> RIGHT JOIN Department
    -> ON Student.Department_ID = Department.Department_ID;
+------------+--------------+---------------+-----------------------+
| Student_ID | Student_Name | Department_ID | Department_Name       |
+------------+--------------+---------------+-----------------------+
|        101 | Aarav Kumar  |             1 | Computer Science      |
|        102 | Diya Sharma  |             1 | Computer Science      |
|        103 | Rahul Reddy  |             2 | Information Technology |
|        104 | Ananya Rao   |             3 | Electronics           |
|        105 | Kiran Singh  |             2 | Information Technology |
|       NULL | NULL         |             4 | Mechanical Engineering |
|       NULL | NULL         |             5 | Civil Engineering     |
+------------+--------------+---------------+-----------------------+
7 rows in set (0.001 sec)
```

5.4.3 LEFT JOIN

Aim / Purpose: To display all students along with their corresponding department names. LEFT JOIN returns all rows from the left table (Student) and the matching rows from Department; if a student has no matching department, the department details appear as NULL.

SQL Query and Output:

```
mysql> SELECT Student.Student_ID,
    ->        Student.Student_Name,
    ->        Student.Department_ID,
    ->        Department.Department_Name
    -> FROM Student
    -> LEFT JOIN Department
    -> ON Student.Department_ID = Department.Department_ID;
+------------+--------------+---------------+-----------------------+
| Student_ID | Student_Name | Department_ID | Department_Name       |
+------------+--------------+---------------+-----------------------+
|        101 | Aarav Kumar  |             1 | Computer Science      |
|        102 | Diya Sharma  |             1 | Computer Science      |
|        103 | Rahul Reddy  |             2 | Information Technology |
|        104 | Ananya Rao   |             3 | Electronics           |
|        105 | Kiran Singh  |             2 | Information Technology |
+------------+--------------+---------------+-----------------------+
5 rows in set (0.004 sec)
```

5.4.4 FULL OUTER JOIN

Aim / Purpose: To display all students and all departments, including records that do not have matching records in the other table. FULL OUTER JOIN returns matching and non-matching rows from both tables. MySQL does not directly support the FULL OUTER JOIN keyword, so LEFT JOIN and RIGHT JOIN are combined using UNION to obtain the full outer join result.

SQL Query and Output:

```
mysql> SELECT Student.Student_ID,
    ->        Student.Student_Name,
    ->        Department.Department_ID,
    ->        Department.Department_Name
    -> FROM Student
    -> LEFT JOIN Department
    -> ON Student.Department_ID = Department.Department_ID
    -> UNION
    -> SELECT Student.Student_ID,
    ->        Student.Student_Name,
    ->        Department.Department_ID,
    ->        Department.Department_Name
    -> FROM Student
    -> RIGHT JOIN Department
    -> ON Student.Department_ID = Department.Department_ID
    -> WHERE Student.Student_ID IS NULL;
+------------+--------------+---------------+-----------------------+
| Student_ID | Student_Name | Department_ID | Department_Name       |
+------------+--------------+---------------+-----------------------+
|        101 | Aarav Kumar  |             1 | Computer Science      |
|        102 | Diya Sharma  |             1 | Computer Science      |
|        103 | Rahul Reddy  |             2 | Information Technology |
|        104 | Ananya Rao   |             3 | Electronics           |
|        105 | Kiran Singh  |             2 | Information Technology |
|       NULL | NULL         |             4 | Mechanical Engineering |
|       NULL | NULL         |             5 | Civil Engineering     |
+------------+--------------+---------------+-----------------------+
7 rows in set (0.002 sec)
```

5.4.5 CROSS JOIN

Aim / Purpose: To combine every student record with every department record. CROSS JOIN produces the Cartesian product of two tables: each student is paired with every department, whether or not the student belongs to it. The Student table contains 5 records and the Department table contains 5 records, so the total number of rows is 5 × 5 = 25.

SQL Query and Output:

```
mysql> SELECT Student.Student_ID,
    ->        Student.Student_Name,
    ->        Department.Department_Name
    -> FROM Student
    -> CROSS JOIN Department
    -> ORDER BY Student.Student_ID,
    ->          Department.Department_ID;
+------------+--------------+-----------------------+
| Student_ID | Student_Name | Department_Name       |
+------------+--------------+-----------------------+
|        101 | Aarav Kumar  | Computer Science      |
|        101 | Aarav Kumar  | Information Technology |
|        101 | Aarav Kumar  | Electronics           |
|        101 | Aarav Kumar  | Mechanical Engineering |
|        101 | Aarav Kumar  | Civil Engineering     |
```

```
|        102 | Diya Sharma  | Computer Science       |
|        102 | Diya Sharma  | Information Technology |
|        102 | Diya Sharma  | Electronics            |
|        102 | Diya Sharma  | Mechanical Engineering |
|        102 | Diya Sharma  | Civil Engineering      |
|        103 | Rahul Reddy  | Computer Science       |
|        103 | Rahul Reddy  | Information Technology |
|        103 | Rahul Reddy  | Electronics            |
|        103 | Rahul Reddy  | Mechanical Engineering |
|        103 | Rahul Reddy  | Civil Engineering      |
|        104 | Ananya Rao   | Computer Science       |
|        104 | Ananya Rao   | Information Technology |
|        104 | Ananya Rao   | Electronics            |
|        104 | Ananya Rao   | Mechanical Engineering |
|        104 | Ananya Rao   | Civil Engineering      |
|        105 | Kiran Singh  | Computer Science       |
|        105 | Kiran Singh  | Information Technology |
|        105 | Kiran Singh  | Electronics            |
|        105 | Kiran Singh  | Mechanical Engineering |
|        105 | Kiran Singh  | Civil Engineering      |
+------------+--------------+------------------------+
25 rows in set (0.000 sec)
```

5.5 GROUP BY and HAVING with Joins

The following queries combine the joins of Section 5.4 with ORDER BY, GROUP BY and HAVING to summarise data that is stored across several tables.

5.5.1 Display Marks from Highest to Lowest

Purpose: Show each student's recorded mark in descending order. JOIN connects marks to enrollment and then to the student; DESC sorts the marks from highest to lowest.

SQL Command and Output:

```
mysql> SELECT s.Student_Name, m.Marks_Obtained
    -> FROM Marks AS m
    -> JOIN Enrollment AS e ON m.Enrollment_ID = e.Enrollment_ID
    -> JOIN Student AS s ON e.Student_ID = s.Student_ID
    -> ORDER BY m.Marks_Obtained DESC;
+--------------+----------------+
| Student_Name | Marks_Obtained |
+--------------+----------------+
| Diya Sharma  |          90.00 |
| Ananya Rao   |          88.00 |
| Aarav Kumar  |          85.00 |
| Rahul Reddy  |          78.00 |
| Kiran Singh  |          72.00 |
+--------------+----------------+
5 rows in set (0.000 sec)
```

Interpretation: Diya Sharma has the highest recorded mark (90), and Kiran Singh has the lowest (72).

5.5.2 Count Students in Each Department

Purpose: Count how many students belong to each department. GROUP BY creates a group for each department; LEFT JOIN includes departments with no matching student, and COUNT(s.Student_ID) counts only matched students.

SQL Command and Output:

```
mysql> SELECT d.Department_Name, COUNT(s.Student_ID) AS Student_Count
    -> FROM Department AS d
    -> LEFT JOIN Student AS s ON d.Department_ID = s.Department_ID
    -> GROUP BY d.Department_ID, d.Department_Name
    -> ORDER BY d.Department_ID;
+-----------------------+---------------+
| Department_Name       | Student_Count |
+-----------------------+---------------+
| Computer Science      |             2 |
| Information Technology |             2 |
| Electronics           |             1 |
| Mechanical Engineering |             0 |
| Civil Engineering     |             0 |
+-----------------------+---------------+
5 rows in set (0.001 sec)
```

Interpretation: Computer Science and Information Technology have two students each; Electronics has one; Mechanical and Civil Engineering have none in this sample.

5.5.3 Average Marks by Course

Purpose: Calculate average recorded marks for each course. The joins link each course to its examinations and marks; GROUP BY creates one group per course and AVG calculates each course's average.

SQL Command and Output:

```
mysql> SELECT c.Course_Name,
    ->        ROUND(AVG(m.Marks_Obtained), 2) AS Average_Marks,
    ->        COUNT(m.Mark_ID) AS Marks_Records
    -> FROM Course AS c
    -> JOIN Examination AS ex ON c.Course_ID = ex.Course_ID
    -> JOIN Marks AS m ON ex.Exam_ID = m.Exam_ID
    -> GROUP BY c.Course_ID, c.Course_Name
    -> ORDER BY Average_Marks DESC;
+--------------------+---------------+---------------+
| Course_Name        | Average_Marks | Marks_Records |
+--------------------+---------------+---------------+
| Digital Electronics |         88.00 |             1 |
| DBMS               |         87.50 |             2 |
| Computer Networks  |         75.00 |             2 |
+--------------------+---------------+---------------+
3 rows in set (0.002 sec)
```

Interpretation: Digital Electronics averages 88 from one record, DBMS averages 87.50 from two records, and Computer Networks averages 75 from two records. Courses without recorded marks are not shown.

5.5.4 Departments with More Than One Student

Purpose: Display only departments whose student count exceeds one. GROUP BY forms department groups, HAVING filters those groups on the count, and ORDER BY sorts the remaining results.

SQL Command and Output:

```
mysql> SELECT d.Department_Name, COUNT(s.Student_ID) AS Student_Count
    -> FROM Department AS d
    -> JOIN Student AS s ON d.Department_ID = s.Department_ID
    -> GROUP BY d.Department_ID, d.Department_Name
    -> HAVING COUNT(s.Student_ID) > 1
    -> ORDER BY Student_Count DESC, d.Department_Name ASC;
+-----------------------+---------------+
| Department_Name       | Student_Count |
+-----------------------+---------------+
| Computer Science      |             2 |
| Information Technology |             2 |
+-----------------------+---------------+
2 rows in set (0.000 sec)
```

5.5.5 Courses with Average Marks Above 80

Purpose: Show courses whose average recorded mark is greater than 80. HAVING keeps only the groups with an average greater than 80.

SQL Command and Output:

```
mysql> SELECT c.Course_Name, ROUND(AVG(m.Marks_Obtained), 2) AS Average_Marks
    -> FROM Course AS c
    -> JOIN Examination AS ex ON c.Course_ID = ex.Course_ID
    -> JOIN Marks AS m ON ex.Exam_ID = m.Exam_ID
    -> GROUP BY c.Course_ID, c.Course_Name
    -> HAVING AVG(m.Marks_Obtained) > 80
    -> ORDER BY Average_Marks DESC;
+--------------------+---------------+
| Course_Name        | Average_Marks |
+--------------------+---------------+
| Digital Electronics |         88.00 |
| DBMS               |         87.50 |
+--------------------+---------------+
2 rows in set (0.003 sec)
```

Interpretation: Digital Electronics and DBMS meet the condition. Computer Networks is excluded because its average is 75.

5.5.6 WHERE and HAVING – Difference

WHERE	HAVING
Filters individual rows before grouping.	Filters groups after GROUP BY.
Example: `WHERE Status = 'Present'`	Example: `HAVING COUNT(*) > 1`
Can reduce which rows enter the grouping operation.	Can test aggregate results such as COUNT() or AVG().

5.5.7 Conclusion

Aggregate functions, ORDER BY, GROUP BY and HAVING help convert stored student information into useful summaries. The queries demonstrate counting students, calculating mark statistics, sorting results, grouping students by department, summarizing attendance and filtering groups based on aggregate conditions.

5.6 Nested Queries and Subqueries

A nested query (subquery) is a query written inside another query. The inner query is executed first and its result is used by the outer query.

5.6.1 Subquery using Student and Department

Aim / Purpose: To display the students who belong to the Computer Science department. The inner query retrieves the Department_ID of Computer Science; the outer query uses that ID to retrieve the matching students.

SQL Query and Output:

```
mysql> SELECT Student_ID, Student_Name
    -> FROM Student
    -> WHERE Department_ID IN
    -> (
    ->     SELECT Department_ID
    ->     FROM Department
    ->     WHERE Department_Name = 'Computer Science'
    -> );
+------------+--------------+
| Student_ID | Student_Name |
+------------+--------------+
|        101 | Aarav Kumar  |
|        102 | Diya Sharma  |
+------------+--------------+
2 rows in set (0.000 sec)
```

5.6.2 Subquery using Course and Faculty

Aim / Purpose: To display the courses taught by faculty members whose designation is Professor. The inner query retrieves the Faculty_ID of Professors; the outer query displays the corresponding courses.

SQL Query and Output:

```
mysql> SELECT Course_ID, Course_Name, Faculty_ID
    -> FROM Course
    -> WHERE Faculty_ID IN
    -> (
    ->     SELECT Faculty_ID
    ->     FROM Faculty
    ->     WHERE Designation = 'Professor'
    -> );
+-----------+--------------------+------------+
| Course_ID | Course_Name        | Faculty_ID |
+-----------+--------------------+------------+
|       301 | DBMS               |        201 |
|       303 | Digital Electronics |        203 |
+-----------+--------------------+------------+
2 rows in set (0.001 sec)
```

5.6.3 Nested Subquery using Student, Enrollment and Marks

Aim / Purpose: To display the students who scored marks greater than the average marks obtained by all students. The innermost query calculates the average of Marks_Obtained; the next query identifies the enrollment records whose marks are greater than that average; the outer query displays the corresponding students.

SQL Query and Output:

```
mysql> SELECT Student_ID, Student_Name
    -> FROM Student
    -> WHERE Student_ID IN
    -> (
    ->     SELECT Student_ID
    ->     FROM Enrollment
    ->     WHERE Enrollment_ID IN
    ->     (
    ->         SELECT Enrollment_ID
    ->         FROM Marks
    ->         WHERE Marks_Obtained >
    ->         (
    ->             SELECT AVG(Marks_Obtained)
    ->             FROM Marks
    ->         )
    ->     )
    -> );
+------------+--------------+
| Student_ID | Student_Name |
+------------+--------------+
|        101 | Aarav Kumar  |
|        102 | Diya Sharma  |
|        104 | Ananya Rao   |
+------------+--------------+
3 rows in set (0.004 sec)
```

5.6.4 Nested Subquery using Student, Enrollment and Attendance

Aim / Purpose: To display the students who were marked absent in attendance. The innermost query retrieves the Enrollment_ID values of Absent records; the next query finds the students for those enrollments; the outer query displays their IDs and names.

SQL Query and Output:

```
mysql> SELECT Student_ID, Student_Name
    -> FROM Student
    -> WHERE Student_ID IN
    -> (
    ->     SELECT Student_ID
    ->     FROM Enrollment
    ->     WHERE Enrollment_ID IN
    ->     (
    ->         SELECT Enrollment_ID
    ->         FROM Attendance
    ->         WHERE Status = 'Absent'
    ->     )
    -> );
+------------+--------------+
| Student_ID | Student_Name |
+------------+--------------+
|        102 | Diya Sharma  |
|        105 | Kiran Singh  |
+------------+--------------+
2 rows in set (0.002 sec)
```

5.7 Stored Procedures

Stored procedures and triggers are database features that store SQL logic inside the database server to automate operations, reuse SQL code and maintain data integrity. This section and the next (5.8) contain 12 practical examples on the tables created in Section 4: six stored procedures (Examples 1–6) and six triggers (Examples 7–12). They must be run in order.

5.7.1 What is a Stored Procedure?

A stored procedure is a named, pre-written group of SQL statements saved in the database and run on demand with the CALL statement. It can accept parameters, declare variables, use conditions and loops, and return result sets or output values. Stored procedures are used because they give:

- **Reusability and less repeated code** – long multi-table queries become one short CALL.
- **Performance** – the logic runs on the server, reducing the SQL sent by the client.
- **Security** – users can be given EXECUTE permission without direct table access.
- **Easy maintenance** – a change is made once, inside the procedure.

5.7.2 Stored Procedure versus a Normal SQL Query

Point	Normal SQL query	Stored procedure
Storage	Typed or sent by the client each time.	Saved in the database as a named object.
Execution	Run directly, e.g. SELECT	Run by name using CALL procedure_name(...).
Content	A single SQL statement.	Many statements with variables, IF conditions and loops.
Parameters	Values are written inside the query.	Accepts IN, OUT and INOUT parameters.
Reuse / maintenance	Copied wherever needed; every copy edited on change.	Written once, called many times, changed once.

5.7.3 Syntax and Parameters

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE procedure_name ( [IN | OUT | INOUT] parameter_name data_type, ... )
    -> BEGIN
    ->     -- one or more SQL statements
    -> END //
mysql> DELIMITER ;
mysql> CALL procedure_name ( argument_values );
mysql> DROP PROCEDURE IF EXISTS procedure_name;
```

Why DELIMITER? The MySQL client treats a semicolon as the end of a statement, but a procedure body contains semicolons. `DELIMITER //` temporarily changes the terminator so the whole block is sent as one statement; `DELIMITER ;` restores it. Each parameter has a mode, a name and a data type:

Mode	Meaning	Used in
IN	Default. Passes a value into the procedure.	Examples 2, 4, 5, 6

Mode	Meaning	Used in
OUT	The procedure assigns a value that is returned to the caller through a user variable.	Example 6
INOUT	Value is passed in, may be changed, and passed back out.	Syntax only

5.7.4 Practical Examples

Example 1: Display All Student Records

Aim: To create a stored procedure that displays every record of the Student table with a single CALL statement.

SQL Code and Output:

```
mysql> DROP PROCEDURE IF EXISTS GetAllStudents;
Query OK, 0 rows affected (0.002 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE GetAllStudents()
    -> BEGIN
    ->     SELECT *
    ->     FROM Student
    ->     ORDER BY Student_ID;
    -> END //
Query OK, 0 rows affected (0.004 sec)

mysql> DELIMITER ;

mysql> CALL GetAllStudents();
+------------+--------------+---------------+------------+--------------------+---------------+
| Student_ID | Student_Name | Date_of_Birth | Phone      | Email              | Department_ID |
+------------+--------------+---------------+------------+--------------------+---------------+
|        101 | Aarav Kumar  | 2005-03-12    | 9876500199 | aarav@example.com  |             1 |
|        102 | Diya Sharma  | 2005-07-24    | 9876500102 | diya@example.com   |             1 |
|        103 | Rahul Reddy  | 2004-11-08    | 9876500103 | rahul@example.com  |             2 |
|        104 | Ananya Rao   | 2005-01-16    | 9876500104 | ananya@example.com |             3 |
|        105 | Kiran Singh  | 2005-09-30    | 9876500105 | kiran@example.com  |             2 |
+------------+--------------+---------------+------------+--------------------+---------------+
5 rows in set (0.001 sec)
Query OK, 0 rows affected (0.000 sec)
```

Explanation: `DELIMITER //` changes the terminator because the procedure body contains a semicolon; `DROP PROCEDURE IF EXISTS` removes an older version so the script can be re-run without an error. **Conclusion:** All 5 student records were returned in order of Student_ID (the phone number of student 101 was updated in Section 5.2).

Example 2: Retrieve Details of a Student by Student ID

Aim: To create a procedure with an IN parameter that returns the details of one student together with the department name.

SQL Code and Output:

```
mysql> DROP PROCEDURE IF EXISTS GetStudentByID;
Query OK, 0 rows affected (0.001 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE GetStudentByID(IN p_student_id INT)
    -> BEGIN
    ->     SELECT s.Student_ID, s.Student_Name, s.Date_of_Birth,
    ->            s.Phone, s.Email, d.Department_Name
    ->     FROM Student s
    ->     JOIN Department d ON s.Department_ID = d.Department_ID
    ->     WHERE s.Student_ID = p_student_id;
    -> END //
Query OK, 0 rows affected (0.004 sec)

mysql> DELIMITER ;

mysql> CALL GetStudentByID(103);
+------------+--------------+---------------+------------+-------------------+-----------------------+
| Student_ID | Student_Name | Date_of_Birth | Phone      | Email             | Department_Name       |
+------------+--------------+---------------+------------+-------------------+-----------------------+
|        103 | Rahul Reddy  | 2004-11-08    | 9876500103 | rahul@example.com | Information Technology |
+------------+--------------+---------------+------------+-------------------+-----------------------+
1 row in set (0.000 sec)
Query OK, 0 rows affected (0.003 sec)
```

Explanation: `IN p_student_id INT` is an input parameter; the value given in the CALL statement is stored in it, and the WHERE clause uses it, so one procedure can return any student. **Conclusion:** Passing 103 returned only Rahul Reddy with his department, Information Technology.

Example 3: Display All Courses with Department and Faculty

Aim: To create a procedure that lists every course with its credits, department and responsible faculty member.

SQL Code and Output:

```
mysql> DROP PROCEDURE IF EXISTS GetAllCourses;
Query OK, 0 rows affected (0.001 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE GetAllCourses()
    -> BEGIN
    ->     SELECT c.Course_ID, c.Course_Name, c.Credits,
    ->            d.Department_Name, f.Faculty_Name
    ->     FROM Course c
    ->     JOIN Department d ON c.Department_ID = d.Department_ID
    ->     JOIN Faculty f    ON c.Faculty_ID    = f.Faculty_ID
    ->     ORDER BY c.Course_ID;
    -> END //
Query OK, 0 rows affected (0.004 sec)

mysql> DELIMITER ;

mysql> CALL GetAllCourses();
+-----------+----------------------+---------+----------------------+--------------+
```

```
| Course_ID | Course_Name            | Credits | Department_Name        | Faculty_Name |
+-----------+------------------------+---------+------------------------+--------------+
|       301 | DBMS                   |       4 | Computer Science       | Dr. Meera    |
|       302 | Computer Networks      |       3 | Information Technology | Mr. Arjun    |
|       303 | Digital Electronics    |       4 | Electronics            | Dr. Kavita   |
|       304 | Engineering Mechanics  |       3 | Mechanical Engineering | Mr. Suresh   |
|       305 | Structural Engineering |       4 | Civil Engineering      | Dr. Priya    |
+-----------+------------------------+---------+------------------------+--------------+
5 rows in set (0.004 sec)
Query OK, 0 rows affected (0.002 sec)
```

Explanation: No parameters are needed because the procedure always returns every course; Course is joined with Department and Faculty through the foreign keys, converting IDs into names. **Conclusion:** All 5 courses were displayed with department and faculty names; the join logic is stored once and reused by a short CALL.

Example 4: Retrieve Marks and Grades of a Student

Aim: To create a procedure that returns every mark and grade of a student together with the course and exam names.

SQL Code and Output:

```
mysql> DROP PROCEDURE IF EXISTS GetStudentMarks;
Query OK, 0 rows affected (0.004 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE GetStudentMarks(IN p_student_id INT)
    -> BEGIN
    ->     SELECT s.Student_Name, c.Course_Name, ex.Exam_Name,
    ->            m.Marks_Obtained, ex.Max_Marks, m.Grade
    ->     FROM Marks m
    ->     JOIN Enrollment en ON m.Enrollment_ID = en.Enrollment_ID
    ->     JOIN Student s ON en.Student_ID = s.Student_ID
    ->     JOIN Course c ON en.Course_ID = c.Course_ID
    ->     JOIN Examination ex ON m.Exam_ID = ex.Exam_ID
    ->     WHERE s.Student_ID = p_student_id;
    -> END //
Query OK, 0 rows affected (0.004 sec)

mysql> DELIMITER ;

mysql> CALL GetStudentMarks(101);
+--------------+-------------+-----------------+----------------+-----------+-------+
| Student_Name | Course_Name | Exam_Name       | Marks_Obtained | Max_Marks | Grade |
+--------------+-------------+-----------------+----------------+-----------+-------+
| Aarav Kumar  | DBMS        | DBMS Internal 1 |          85.00 |    100.00 | A     |
+--------------+-------------+-----------------+----------------+-----------+-------+
1 row in set (0.002 sec)
Query OK, 0 rows affected (0.000 sec)

mysql> CALL GetStudentMarks(105);
+--------------+------------------+--------------------+----------------+-----------+-------+
| Student_Name | Course_Name      | Exam_Name          | Marks_Obtained | Max_Marks | Grade |
+--------------+------------------+--------------------+----------------+-----------+-------+
| Kiran Singh  | Computer Networks | Networks Internal 2 |          72.00 |    100.00 | B     |
+--------------+------------------+--------------------+----------------+-----------+-------+
1 row in set (0.003 sec)
Query OK, 0 rows affected (0.001 sec)
```

Explanation: Marks stores Enrollment_ID, not the student, so the query follows Marks → Enrollment → Student; Enrollment supplies the course and Examination supplies the exam name and maximum marks. **Conclusion:** The procedure returned 85 (grade A) for Aarav Kumar and 72 (grade B) for Kiran Singh; a five-table join is hidden behind one CALL.

Example 5: Display Attendance Details of a Student

Aim: To create a procedure that displays the attendance records of a student for every enrolled course.

SQL Code and Output:

```
mysql> DROP PROCEDURE IF EXISTS GetStudentAttendance;
Query OK, 0 rows affected (0.002 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE GetStudentAttendance(IN p_student_id INT)
    -> BEGIN
    ->     SELECT s.Student_Name, c.Course_Name,
    ->            a.Attendance_Date, a.Status, a.Remarks
    ->     FROM Attendance a
    ->     JOIN Enrollment en ON a.Enrollment_ID = en.Enrollment_ID
    ->     JOIN Student s     ON en.Student_ID   = s.Student_ID
    ->     JOIN Course c      ON en.Course_ID    = c.Course_ID
    ->     WHERE s.Student_ID = p_student_id
    ->     ORDER BY a.Attendance_Date;
    -> END //
Query OK, 0 rows affected (0.004 sec)

mysql> DELIMITER ;

mysql> CALL GetStudentAttendance(102);
+--------------+-------------+-----------------+--------+-------------------+
| Student_Name | Course_Name | Attendance_Date | Status | Remarks           |
+--------------+-------------+-----------------+--------+-------------------+
| Diya Sharma  | DBMS        | 2026-07-10      | Absent | Absent from class |
+--------------+-------------+-----------------+--------+-------------------+
1 row in set (0.002 sec)
Query OK, 0 rows affected (0.001 sec)
```

Explanation: Attendance is linked to a student through Enrollment, so four tables are joined and results are sorted by Attendance_Date. **Conclusion:** Diya Sharma was Absent in DBMS on 2026-07-10; calling the procedure with another Student_ID shows that student's attendance.

Example 6: Count Students in a Department (OUT Parameter)

Aim: To demonstrate a procedure that uses an IN parameter (department ID) and an OUT parameter returning a calculated value.

SQL Code and Output:

```
mysql> DROP PROCEDURE IF EXISTS GetDeptStudentCount;
Query OK, 0 rows affected (0.003 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE GetDeptStudentCount(IN p_dept_id INT,
    ->                                      OUT p_total INT)
    -> BEGIN
    ->     SELECT COUNT(*) INTO p_total
    ->     FROM Student
    ->     WHERE Department_ID = p_dept_id;
    -> END //
Query OK, 0 rows affected (0.004 sec)

mysql> DELIMITER ;

mysql> CALL GetDeptStudentCount(1, @total);
Query OK, 1 row affected (0.000 sec)

mysql> SELECT @total AS Total_Students;
+----------------+
| Total_Students |
+----------------+
|              2 |
+----------------+
1 row in set (0.000 sec)
```

Explanation: `IN p_dept_id` receives the department number; `OUT p_total` sends a value back to the caller, and `SELECT COUNT(*) INTO p_total` stores the count in it instead of displaying a table. The caller passes a user variable (`@total`) and reads it afterwards with a SELECT. **Conclusion:** Department 1 (Computer Science) has 2 students; the value came back through the OUT parameter, which exists only in the current session.

5.8 Triggers and Data Validation

5.8.1 What is a Trigger?

A trigger is a stored program attached to a table that the database executes **automatically** when an INSERT, UPDATE or DELETE event happens on that table. Unlike a procedure it is never called with CALL: the user performs the normal data operation and the trigger fires by itself. Triggers are used for:

- **Data validation** – rejecting invalid values such as marks outside the allowed range.
- **Auditing** – recording what changed, with old and new values.
- **Integrity rules** – enforcing business rules that simple constraints cannot express.
- **Consistency** – the rule applies no matter which user or application changes the data.

5.8.2 BEFORE and AFTER Triggers, Events and Execution

A BEFORE trigger runs before the row change and can stop it by raising an error; an AFTER trigger runs once the change has succeeded. Combining the two timings with the three events gives six trigger types, all demonstrated in this chapter:

Event	BEFORE trigger	AFTER trigger	Values
INSERT	Example 7 – validate new marks	Example 8 – log new student	NEW
UPDATE	Example 9 – validate changed marks	Example 10 – audit changes	OLD, NEW
DELETE	Example 11 – block pending fee deletion	Example 12 – log deleted student	OLD

Execution order: SQL statement (INSERT / UPDATE / DELETE) → BEFORE trigger runs → row is inserted / updated / deleted → AFTER trigger runs → statement completes. Inside a trigger, `OLD.column` is the value before the change and `NEW.column` the value after (or about to be written by) the change.

5.8.3 Basic Syntax of a Trigger

```
mysql> DELIMITER //
mysql> CREATE TRIGGER trigger_name
    -> { BEFORE | AFTER } { INSERT | UPDATE | DELETE } ON table_name
    -> FOR EACH ROW
    -> BEGIN
    ->     -- statements using OLD.column and/or NEW.column
    ->     -- SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = '...';  (raises an error)
    -> END //
mysql> DELIMITER ;
mysql> SHOW TRIGGERS;
mysql> DROP TRIGGER IF EXISTS trigger_name;
```

`SIGNAL` raises a custom error inside a trigger. SQLSTATE '45000' means “unhandled user-defined exception”; the error cancels the statement that fired the trigger and MESSAGE_TEXT is shown to the user.

5.8.4 Practical Examples

Example 7: Validate Marks Before Inserting (BEFORE INSERT)

Aim: To reject any new Marks record whose Marks_Obtained is negative or greater than the maximum marks of the exam.

SQL Code and Output:

```
mysql> DROP TRIGGER IF EXISTS trg_marks_before_insert;
Query OK, 0 rows affected (0.001 sec)

mysql> DELIMITER //
mysql> CREATE TRIGGER trg_marks_before_insert
    -> BEFORE INSERT ON Marks
    -> FOR EACH ROW
    -> BEGIN
    ->     DECLARE v_max DECIMAL(5,2);
    ->     SELECT Max_Marks INTO v_max
    ->     FROM Examination
    ->     WHERE Exam_ID = NEW.Exam_ID;
    ->     IF NEW.Marks_Obtained < 0 OR NEW.Marks_Obtained > v_max THEN
    ->         SIGNAL SQLSTATE '45000'
```

```
    ->         SET MESSAGE_TEXT = 'Invalid marks: must be 0 to Max_Marks';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.006 sec)

mysql> DELIMITER ;

mysql> INSERT INTO Marks VALUES (606, 402, 501, 82, 'A', 'Good performance');
Query OK, 1 row affected (0.002 sec)

mysql> INSERT INTO Marks VALUES (607, 401, 502, 150, 'A+', 'Invalid test');
ERROR 1644 (45000): Invalid marks: must be 0 to Max_Marks

mysql> SELECT Mark_ID, Enrollment_ID, Marks_Obtained, Grade FROM Marks WHERE Mark_ID IN (606, 607);
+---------+--------------+----------------+-------+
| Mark_ID | Enrollment_ID | Marks_Obtained | Grade |
+---------+--------------+----------------+-------+
|     606 |          402 |          82.00 | A     |
+---------+--------------+----------------+-------+
1 row in set (0.004 sec)
```

Explanation: BEFORE INSERT runs just before the row is written, so it can stop a bad row from being stored; `NEW.Exam_ID` and `NEW.Marks_Obtained` are the values of the row about to be inserted, and `SIGNAL SQLSTATE '45000'` cancels the INSERT. **Conclusion:** The valid row was stored, while the row with 150 marks was blocked with ERROR 1644 and not saved.

Example 8: Log New Students in an Audit Table (AFTER INSERT)

Aim: To record every newly inserted student automatically in a separate audit table (supporting table Student_Audit).

Supporting Table (create first), Trigger and Output:

```
mysql> CREATE TABLE Student_Audit (
    ->     Audit_ID          INT AUTO_INCREMENT PRIMARY KEY,
    ->     Student_ID        INT NOT NULL,
    ->     Action_Type       VARCHAR(10) NOT NULL,
    ->     Old_Name          VARCHAR(100),
    ->     New_Name          VARCHAR(100),
    ->     Old_Phone         VARCHAR(15),
    ->     New_Phone         VARCHAR(15),
    ->     Old_Email         VARCHAR(100),
    ->     New_Email         VARCHAR(100),
    ->     Old_Department_ID INT,
    ->     New_Department_ID INT,
    ->     Changed_At        DATETIME DEFAULT CURRENT_TIMESTAMP
    -> );
Query OK, 0 rows affected (0.021 sec)

mysql> DROP TRIGGER IF EXISTS trg_student_after_insert;
Query OK, 0 rows affected (0.002 sec)

mysql> DELIMITER //
mysql> CREATE TRIGGER trg_student_after_insert
    -> AFTER INSERT ON Student
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO Student_Audit
    ->         (Student_ID, Action_Type, New_Name,
    ->          New_Phone, New_Email, New_Department_ID)
    ->     VALUES
    ->         (NEW.Student_ID, 'INSERT', NEW.Student_Name,
    ->          NEW.Phone, NEW.Email, NEW.Department_ID);
```

```
    -> END //
Query OK, 0 rows affected (0.006 sec)

mysql> DELIMITER ;

mysql> INSERT INTO Student (Student_ID, Student_Name, Date_of_Birth, Phone, Email, Department_ID)
    -> VALUES (106, 'Sneha Patel', '2005-05-14', '9876500106', 'sneha@example.com', 1);
Query OK, 1 row affected (0.002 sec)

mysql> SELECT Audit_ID, Student_ID, Action_Type, New_Name,
    ->        New_Phone, New_Department_ID
    -> FROM Student_Audit;
+----------+------------+-------------+-------------+------------+------------------+
| Audit_ID | Student_ID | Action_Type | New_Name    | New_Phone  | New_Department_ID |
+----------+------------+-------------+-------------+------------+------------------+
|        1 |        106 | INSERT      | Sneha Patel | 9876500106 |                1 |
+----------+------------+-------------+-------------+------------+------------------+
1 row in set (0.000 sec)
```

Explanation: Student_Audit has no foreign key to Student on purpose, so audit rows survive even if a student is deleted. AFTER INSERT runs once the row has been stored, which is the right moment to write a log entry; for an INSERT only NEW values exist. **Conclusion:** One INSERT on Student automatically created an audit row of type INSERT holding the new student's details.

Example 9: Prevent Invalid Marks During Update (BEFORE UPDATE)

Aim: To stop existing marks from being changed to a value outside the allowed range.

SQL Code and Output:

```
mysql> DROP TRIGGER IF EXISTS trg_marks_before_update;
Query OK, 0 rows affected (0.001 sec)

mysql> DELIMITER //
mysql> CREATE TRIGGER trg_marks_before_update
    -> BEFORE UPDATE ON Marks
    -> FOR EACH ROW
    -> BEGIN
    ->     DECLARE v_max DECIMAL(5,2);
    ->     SELECT Max_Marks INTO v_max
    ->     FROM Examination
    ->     WHERE Exam_ID = NEW.Exam_ID;
    ->     IF NEW.Marks_Obtained < 0 OR NEW.Marks_Obtained > v_max THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Invalid update: marks out of range';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.006 sec)

mysql> DELIMITER ;

mysql> UPDATE Marks SET Marks_Obtained = 120 WHERE Mark_ID = 601;
ERROR 1644 (45000): Invalid update: marks out of range

mysql> UPDATE Marks SET Marks_Obtained = 88 WHERE Mark_ID = 601;
Query OK, 1 row affected (0.002 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT Mark_ID, Marks_Obtained, Grade FROM Marks WHERE Mark_ID = 601;
+---------+---------------+-------+
| Mark_ID | Marks_Obtained | Grade |
+---------+---------------+-------+
|     601 |          88.00 | A     |
```

```
+---------+---------------+-------+
1 row in set (0.000 sec)
```

Explanation: Example 7 protects new rows only; a later UPDATE could still store a wrong value, so a second trigger is needed. In a BEFORE UPDATE trigger OLD holds the existing row and NEW the values about to be written; the check uses NEW. **Conclusion:** The update to 120 was rejected, while the update to 88 succeeded; Marks is now protected for both INSERT and UPDATE.

Example 10: Audit Changes to Student Details (AFTER UPDATE)

Aim: To record every change to a student's name, phone, email or department, storing both old and new values.

SQL Code and Output:

```
mysql> DROP TRIGGER IF EXISTS trg_student_after_update;
Query OK, 0 rows affected (0.002 sec)

mysql> DELIMITER //
mysql> CREATE TRIGGER trg_student_after_update
    -> AFTER UPDATE ON Student
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NOT (OLD.Student_Name  <=> NEW.Student_Name
    ->         AND OLD.Phone         <=> NEW.Phone
    ->         AND OLD.Email         <=> NEW.Email
    ->         AND OLD.Department_ID <=> NEW.Department_ID) THEN
    ->         INSERT INTO Student_Audit
    ->             (Student_ID, Action_Type, Old_Name, New_Name, Old_Phone,
    ->              New_Phone, Old_Email, New_Email, Old_Department_ID,
    ->              New_Department_ID)
    ->         VALUES
    ->             (OLD.Student_ID, 'UPDATE', OLD.Student_Name, NEW.Student_Name,
    ->              OLD.Phone, NEW.Phone, OLD.Email, NEW.Email,
    ->              OLD.Department_ID, NEW.Department_ID);
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.006 sec)

mysql> DELIMITER ;

mysql> UPDATE Student SET Phone = '9876500188' WHERE Student_ID = 101;
Query OK, 1 row affected (0.001 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT Audit_ID, Student_ID, Action_Type, Old_Phone,
    ->        New_Phone, Changed_At
    -> FROM Student_Audit
    -> WHERE Action_Type = 'UPDATE';
+----------+------------+-------------+------------+------------+---------------------+
| Audit_ID | Student_ID | Action_Type | Old_Phone  | New_Phone  | Changed_At          |
+----------+------------+-------------+------------+------------+---------------------+
|        2 |        101 | UPDATE      | 9876500199 | 9876500188 | 2026-10-21 07:45:18 |
+----------+------------+-------------+------------+------------+---------------------+
1 row in set (0.000 sec)
```

Explanation: AFTER UPDATE runs once the update has succeeded; the null-safe operator <=> compares values even when a column is NULL, so a row is logged only when a tracked column

really changed. **Conclusion:** A single UPDATE produced an audit row with the old phone 9876500199, the new phone 9876500188 and the time of change.

Example 11: Block Deletion of Pending Fee Records (BEFORE DELETE)

Aim: To prevent a Fee record from being deleted while its Payment_Status is Pending.

SQL Code and Output:

```
mysql> DROP TRIGGER IF EXISTS trg_fee_before_delete;
Query OK, 0 rows affected (0.000 sec)

mysql> DELIMITER //
mysql> CREATE TRIGGER trg_fee_before_delete
    -> BEFORE DELETE ON Fee
    -> FOR EACH ROW
    -> BEGIN
    ->     IF OLD.Payment_Status = 'Pending' THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Pending fee records cannot be deleted';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.006 sec)

mysql> DELIMITER ;

mysql> DELETE FROM Fee WHERE Fee_ID = 802;
ERROR 1644 (45000): Pending fee records cannot be deleted

mysql> INSERT INTO Fee VALUES (806, 105, 1000.00, '2026-07-15', '2026-07-12', 'Paid', 'Cash');
Query OK, 1 row affected (0.001 sec)

mysql> DELETE FROM Fee WHERE Fee_ID = 806;
Query OK, 1 row affected (0.001 sec)

mysql> SELECT Fee_ID, Student_ID, Amount, Payment_Status FROM Fee WHERE Fee_ID IN (802, 806);
+--------+------------+----------+----------------+
| Fee_ID | Student_ID | Amount   | Payment_Status |
+--------+------------+----------+----------------+
|    802 |        102 | 45000.00 | Pending        |
+--------+------------+----------+----------------+
1 row in set (0.000 sec)
```

Explanation: BEFORE DELETE runs before the row is removed; for a DELETE only OLD values exist. If the old status is Pending, SIGNAL cancels the DELETE; a temporary paid record (806) shows that other deletions still work. **Conclusion:** The pending record 802 could not be deleted (ERROR 1644), while the paid record 806 was removed normally.

Example 12: Log Deleted Student Records (AFTER DELETE)

Aim: To keep a permanent record of any deleted student by copying the deleted row into Student_Audit.

SQL Code and Output:

```
mysql> DROP TRIGGER IF EXISTS trg_student_after_delete;
Query OK, 0 rows affected (0.003 sec)

mysql> DELIMITER //
mysql> CREATE TRIGGER trg_student_after_delete
    -> AFTER DELETE ON Student
    -> FOR EACH ROW
    -> BEGIN
```

```
    ->     INSERT INTO Student_Audit
    ->         (Student_ID, Action_Type, Old_Name,
    ->          Old_Phone, Old_Email, Old_Department_ID)
    ->     VALUES
    ->         (OLD.Student_ID, 'DELETE', OLD.Student_Name,
    ->          OLD.Phone, OLD.Email, OLD.Department_ID);
    -> END //
Query OK, 0 rows affected (0.006 sec)

mysql> DELIMITER ;

mysql> DELETE FROM Student WHERE Student_ID = 106;
Query OK, 1 row affected (0.002 sec)

mysql> SELECT * FROM Student WHERE Student_ID = 106;
Empty set (0.000 sec)

mysql> SELECT Audit_ID, Student_ID, Action_Type, Old_Name,
    ->        Old_Phone, Old_Department_ID
    -> FROM Student_Audit
    -> WHERE Action_Type = 'DELETE';
+----------+------------+-------------+-------------+------------+-------------------+
| Audit_ID | Student_ID | Action_Type | Old_Name    | Old_Phone  | Old_Department_ID |
+----------+------------+-------------+-------------+------------+-------------------+
|        3 |        106 | DELETE      | Sneha Patel | 9876500106 |                 1 |
+----------+------------+-------------+-------------+------------+-------------------+
1 row in set (0.000 sec)
```

Explanation: AFTER DELETE fires after the student row has been removed; only OLD values exist, so the trigger saves them. The temporary student 106 inserted in Example 8 is deleted, exactly like the DML commands of Section 5.2. **Conclusion:** Student 106 no longer exists in Student, but the audit table keeps a DELETE entry with the full details of Sneha Patel.

5.8.5 Summary and Verification

Stored procedures simplified frequent operations by turning multi-table joins into short CALL statements, and triggers added automatic protection and traceability. The listing below shows the objects created in Sections 5.7 and 5.8 and the complete audit trail recorded by the triggers.

Output:

```
mysql> SELECT ROUTINE_NAME AS Procedure_Name, ROUTINE_TYPE
    -> FROM information_schema.ROUTINES
    -> WHERE ROUTINE_SCHEMA = 'StudentInformationSystem'
    -> ORDER BY ROUTINE_NAME;
+----------------------+--------------+
| Procedure_Name       | ROUTINE_TYPE |
+----------------------+--------------+
| GetAllCourses        | PROCEDURE    |
| GetAllStudents       | PROCEDURE    |
| GetDeptStudentCount  | PROCEDURE    |
| GetStudentAttendance | PROCEDURE    |
| GetStudentByID       | PROCEDURE    |
| GetStudentMarks      | PROCEDURE    |
+----------------------+--------------+
6 rows in set (0.255 sec)

mysql> SELECT TRIGGER_NAME, ACTION_TIMING, EVENT_MANIPULATION, EVENT_OBJECT_TABLE
    -> FROM information_schema.TRIGGERS
    -> WHERE TRIGGER_SCHEMA = 'StudentInformationSystem'
    -> ORDER BY TRIGGER_NAME;
+-------------------------+---------------+--------------------+--------------------+
```

```
| TRIGGER_NAME             | ACTION_TIMING | EVENT_MANIPULATION | EVENT_OBJECT_TABLE |
+--------------------------+---------------+--------------------+--------------------+
| trg_fee_before_delete    | BEFORE        | DELETE             | Fee                |
| trg_marks_before_insert  | BEFORE        | INSERT             | Marks              |
| trg_marks_before_update  | BEFORE        | UPDATE             | Marks              |
| trg_student_after_delete | AFTER         | DELETE             | Student            |
| trg_student_after_insert | AFTER         | INSERT             | Student            |
| trg_student_after_update | AFTER         | UPDATE             | Student            |
+--------------------------+---------------+--------------------+--------------------+
6 rows in set (0.003 sec)

mysql> SELECT Audit_ID, Student_ID, Action_Type, Old_Phone, New_Phone, Changed_At
    -> FROM Student_Audit
    -> ORDER BY Audit_ID;
+----------+------------+-------------+------------+------------+---------------------+
| Audit_ID | Student_ID | Action_Type | Old_Phone  | New_Phone  | Changed_At          |
+----------+------------+-------------+------------+------------+---------------------+
|        1 |        106 | INSERT      | NULL       | 9876500106 | 2026-10-21 07:45:17 |
|        2 |        101 | UPDATE      | 9876500199 | 9876500188 | 2026-10-21 07:45:18 |
|        3 |        106 | DELETE      | 9876500106 | NULL       | 2026-10-21 07:45:18 |
+----------+------------+-------------+------------+------------+---------------------+
3 rows in set (0.000 sec)
```

6. Views

6.1 What is a View?

A **view** is a virtual table defined by a stored SELECT query. It has rows and columns like a table, but it does not store data of its own: every time the view is queried, MySQL runs the underlying SELECT on the base tables and returns the current data. A view can show selected columns of one table, selected rows, or the joined result of several tables, and it can be used in SELECT statements just like a table.

6.2 Advantages of Views

Advantage	Explanation
Simplicity	A long multi-table join is written once in the view; users then run a simple SELECT on the view.
Security	A view exposes only chosen columns and rows (for example no phone number or password hash), and users can be given access to the view instead of the base table.
Logical data independence	Applications that use a view are not affected if the structure of the base tables changes, as long as the view definition is adjusted.
Consistency	Everyone sees the same calculated or joined result, and the data is always current because it is not copied.
Integrity (WITH CHECK OPTION)	Rows inserted or updated through a view must satisfy the view's WHERE condition.

6.3 Syntax of View Commands

General forms:

```
mysql> CREATE VIEW view_name AS
    -> SELECT column_list FROM table_name [WHERE condition];

mysql> CREATE OR REPLACE VIEW view_name AS SELECT ... ;

mysql> CREATE VIEW view_name AS SELECT ... WHERE condition WITH CHECK OPTION;

mysql> SHOW FULL TABLES WHERE Table_type = 'VIEW';
mysql> DROP VIEW [IF EXISTS] view_name;
```

A view is **updatable** (INSERT, UPDATE and DELETE are allowed through it) only when each view row maps to exactly one base-table row. A view is **not updatable** if it uses aggregate functions (COUNT, SUM, AVG, MIN, MAX), GROUP BY, HAVING, DISTINCT, UNION or certain subqueries. Views that join several tables can be queried freely, but changes through them are restricted.

6.4 Practical Examples

The examples are run on the StudentInformationSystem database after the triggers of Section 5.8 (so Mark 601 is 88 and Mark 606 exists). Each CREATE VIEW is followed by a SELECT on the view.

Example 1: Join View – Students with Department Names

Aim: To create a view that hides the join between Student and Department and shows each student with the department name.

SQL Command and Output:

```
mysql> CREATE VIEW Student_Department_View AS
    -> SELECT s.Student_ID, s.Student_Name, s.Email, d.Department_Name
    -> FROM Student s
    -> JOIN Department d ON s.Department_ID = d.Department_ID;
Query OK, 0 rows affected (0.012 sec)

mysql> SELECT * FROM Student_Department_View;
+------------+--------------+--------------------+-----------------------+
| Student_ID | Student_Name | Email              | Department_Name       |
+------------+--------------+--------------------+-----------------------+
|        101 | Aarav Kumar  | aarav@example.com  | Computer Science      |
|        102 | Diya Sharma  | diya@example.com   | Computer Science      |
|        103 | Rahul Reddy  | rahul@example.com  | Information Technology |
|        104 | Ananya Rao   | ananya@example.com | Electronics           |
|        105 | Kiran Singh  | kiran@example.com  | Information Technology |
+------------+--------------+--------------------+-----------------------+
5 rows in set (0.001 sec)
```

Explanation: The view stores only the query. Users select from `Student_Department_View` without knowing about the join, and the phone number and date of birth are not exposed.

Example 2: Row and Column Restricting View with CHECK OPTION

Aim: To create a view that shows only Computer Science students (Department_ID = 1) and only selected columns.

SQL Command and Output:

```
mysql> CREATE VIEW CS_Students AS
    -> SELECT Student_ID, Student_Name, Date_of_Birth, Email, Department_ID
    -> FROM Student
    -> WHERE Department_ID = 1
    -> WITH CHECK OPTION;
Query OK, 0 rows affected (0.012 sec)

mysql> SELECT * FROM CS_Students;
+------------+--------------+---------------+-------------------+---------------+
| Student_ID | Student_Name | Date_of_Birth | Email             | Department_ID |
+------------+--------------+---------------+-------------------+---------------+
|        101 | Aarav Kumar  | 2005-03-12    | aarav@example.com |             1 |
|        102 | Diya Sharma  | 2005-07-24    | diya@example.com  |             1 |
+------------+--------------+---------------+-------------------+---------------+
2 rows in set (0.000 sec)
```

Explanation: The WHERE clause limits the rows and the column list limits the columns (Phone is hidden). `WITH CHECK OPTION` makes MySQL reject any INSERT or UPDATE through the view that would create a row outside Department_ID = 1 (see Example 7).

Example 3: Aggregate View – Students per Department

Aim: To store a department-wise student count as a view.

SQL Command and Output:

```
mysql> CREATE VIEW Dept_Student_Count AS
    -> SELECT d.Department_ID, d.Department_Name,
    ->        COUNT(s.Student_ID) AS Student_Count
    -> FROM Department d
    -> LEFT JOIN Student s ON d.Department_ID = s.Department_ID
    -> GROUP BY d.Department_ID, d.Department_Name;
Query OK, 0 rows affected (0.012 sec)

mysql> SELECT * FROM Dept_Student_Count;
+---------------+-----------------------+---------------+
| Department_ID | Department_Name       | Student_Count |
+---------------+-----------------------+---------------+
|             1 | Computer Science      |             2 |
|             2 | Information Technology |             2 |
|             3 | Electronics           |             1 |
|             4 | Mechanical Engineering |             0 |
|             5 | Civil Engineering     |             0 |
+---------------+-----------------------+---------------+
5 rows in set (0.001 sec)
```

Explanation: The aggregate query of Section 5.5.2 is now available as a one-line SELECT, and the counts always reflect the current Student table.

Example 4: Multi-table View – Student Marks Report

Aim: To create a view that joins Marks, Enrollment, Student, Course and Examination into a ready-made marks report.

SQL Command and Output:

```
mysql> CREATE VIEW Student_Marks_View AS
    -> SELECT s.Student_Name, c.Course_Name, ex.Exam_Name,
    ->        m.Marks_Obtained, m.Grade
    -> FROM Marks m
    -> JOIN Enrollment en ON m.Enrollment_ID = en.Enrollment_ID
    -> JOIN Student s     ON en.Student_ID   = s.Student_ID
    -> JOIN Course c      ON en.Course_ID    = c.Course_ID
    -> JOIN Examination ex ON m.Exam_ID      = ex.Exam_ID;
Query OK, 0 rows affected (0.014 sec)

mysql> SELECT * FROM Student_Marks_View
    -> ORDER BY Marks_Obtained DESC, Student_Name;
+--------------+--------------------+-----------------------+----------------+-------+
| Student_Name | Course_Name        | Exam_Name             | Marks_Obtained | Grade |
+--------------+--------------------+-----------------------+----------------+-------+
| Diya Sharma  | DBMS               | DBMS Internal 2       |          90.00 | A+    |
| Aarav Kumar  | DBMS               | DBMS Internal 1       |          88.00 | A     |
| Ananya Rao   | Digital Electronics | Electronics Internal 1 |          88.00 | A     |
| Diya Sharma  | DBMS               | DBMS Internal 1       |          82.00 | A     |
| Rahul Reddy  | Computer Networks  | Networks Internal 1   |          78.00 | B+    |
| Kiran Singh  | Computer Networks  | Networks Internal 2   |          72.00 | B     |
+--------------+--------------------+-----------------------+----------------+-------+
6 rows in set (0.001 sec)
```

Explanation: A five-table join is reduced to one SELECT. ORDER BY is applied when the view is queried, so the same view can be sorted in different ways.

Example 5: Filtered View – Pending Fees

Aim: To list only the students whose fee payment is still pending.

SQL Command and Output:

```
mysql> CREATE VIEW Pending_Fee_View AS
    -> SELECT f.Fee_ID, s.Student_Name, f.Amount
    -> FROM Fee f
    -> JOIN Student s ON f.Student_ID = s.Student_ID
    -> WHERE f.Payment_Status = 'Pending';
Query OK, 0 rows affected (0.012 sec)

mysql> SELECT * FROM Pending_Fee_View;
+--------+--------------+----------+
| Fee_ID | Student_Name | Amount   |
+--------+--------------+----------+
|    802 | Diya Sharma  | 45000.00 |
|    804 | Ananya Rao   | 48000.00 |
+--------+--------------+----------+
2 rows in set (0.000 sec)
```

Example 6: Updating Data Through a View

Aim: To change a student's email through the updatable view CS_Students and confirm that the base table is changed.

SQL Command and Output:

```
mysql> UPDATE CS_Students
    -> SET Email = 'aarav.kumar@example.com'
    -> WHERE Student_ID = 101;
Query OK, 1 row affected (0.003 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT Student_ID, Student_Name, Email
    -> FROM Student
    -> WHERE Student_ID = 101;
+------------+--------------+------------------------+
| Student_ID | Student_Name | Email                  |
+------------+--------------+------------------------+
|        101 | Aarav Kumar  | aarav.kumar@example.com |
+------------+--------------+------------------------+
1 row in set (0.000 sec)
```

Explanation: CS_Students maps each row to exactly one row of Student, so the UPDATE is applied to the base table. (The audit trigger of Example 10 also records this change in Student_Audit.)

Example 7: WITH CHECK OPTION in Action

Aim: To show that a row outside the view's condition cannot be inserted through CS_Students.

SQL Command and Output:

```
mysql> INSERT INTO CS_Students
    -> (Student_ID, Student_Name, Date_of_Birth, Email, Department_ID)
    -> VALUES
    -> (107, 'Test Student', '2005-01-01', 'test@example.com', 2);
ERROR 1369 (HY000): CHECK OPTION failed 'StudentInformationSystem.CS_Students'
```

Explanation: The new row has Department_ID = 2, which does not satisfy `Department_ID = 1`, so MySQL rejects the INSERT and nothing is stored.

Example 8: A View That Cannot Be Updated

Aim: To show that a view built with GROUP BY and COUNT() is read-only.

SQL Command and Output:

```
mysql> UPDATE Dept_Student_Count
    -> SET Student_Count = 5
    -> WHERE Department_Name = 'Computer Science';
ERROR 1288 (HY000): The target table Dept_Student_Count of the UPDATE is not updatable
```

Explanation: Each row of Dept_Student_Count is calculated from many Student rows, so it cannot be mapped back to a single base-table row.

Example 9: Modifying a View with CREATE OR REPLACE

Aim: To change the definition of Pending_Fee_View so that it also shows the due date and status.

SQL Command and Output:

```
mysql> CREATE OR REPLACE VIEW Pending_Fee_View AS
    -> SELECT f.Fee_ID, s.Student_Name, f.Amount,
    ->        f.Due_Date, f.Payment_Status
    -> FROM Fee f
    -> JOIN Student s ON f.Student_ID = s.Student_ID
    -> WHERE f.Payment_Status = 'Pending';
Query OK, 0 rows affected (0.009 sec)

mysql> SELECT * FROM Pending_Fee_View;
+--------+--------------+----------+------------+----------------+
| Fee_ID | Student_Name | Amount   | Due_Date   | Payment_Status |
+--------+--------------+----------+------------+----------------+
|    802 | Diya Sharma  | 45000.00 | 2026-07-15 | Pending        |
|    804 | Ananya Rao   | 48000.00 | 2026-07-15 | Pending        |
+--------+--------------+----------+------------+----------------+
2 rows in set (0.000 sec)
```

Example 10: Listing and Dropping Views

Aim: To list all views in the database and remove a view that is no longer needed.

SQL Command and Output:

```
mysql> CREATE VIEW Temp_Dept_View AS
    -> SELECT * FROM Department;
Query OK, 0 rows affected (0.008 sec)

mysql> SHOW FULL TABLES WHERE Table_type = 'VIEW';
+-----------------------------------+------------+
| Tables_in_StudentInformationSystem | Table_type |
+-----------------------------------+------------+
| CS_Students                       | VIEW       |
| Dept_Student_Count                | VIEW       |
| Pending_Fee_View                  | VIEW       |
| Student_Department_View           | VIEW       |
| Student_Marks_View                | VIEW       |
| Temp_Dept_View                    | VIEW       |
+-----------------------------------+------------+
6 rows in set (0.002 sec)
```

```
mysql> DROP VIEW Temp_Dept_View;
Query OK, 0 rows affected (0.005 sec)

mysql> SHOW FULL TABLES WHERE Table_type = 'VIEW';
+-----------------------------------+------------+
| Tables_in_StudentInformationSystem | Table_type |
+-----------------------------------+------------+
| CS_Students                       | VIEW       |
| Dept_Student_Count                | VIEW       |
| Pending_Fee_View                  | VIEW       |
| Student_Department_View           | VIEW       |
| Student_Marks_View                | VIEW       |
+-----------------------------------+------------+
5 rows in set (0.002 sec)
```

Explanation: DROP VIEW removes only the view definition; the data in the base tables is not affected.

6.5 Conclusion

Views make the Student Information System easier and safer to use: complex joins and aggregates become simple SELECT statements, sensitive columns can be hidden, and `WITH CHECK OPTION` protects the rules of a restricted view. Views do not store data, so they always show the current contents of the base tables.

7. Database Security and Transaction Management

Besides defining and manipulating data, a DBMS must control **who** may access the data and **how** groups of changes are saved. DCL (Data Control Language) commands manage user permissions, and TCL (Transaction Control Language) commands manage transactions. Both are demonstrated on the StudentInformationSystem database using the Department table.

7.1 Data Control Language (DCL): GRANT and REVOKE

Definition: DCL commands are used to give or remove permissions that allow users to access and work with database objects. They protect the database by making sure each user can perform only the operations he or she is allowed to perform. MySQL provides two main DCL commands: GRANT and REVOKE.

Command	Purpose	General syntax
GRANT	Gives a user permission to perform an operation on a database object.	GRANT privilege ON db.table TO 'user'@'host';
REVOKE	Removes a permission that was previously granted to a user.	REVOKE privilege ON db.table FROM 'user'@'host';

Note: The user must exist before GRANT or REVOKE is run, so a MySQL user `student_user` is created first. The exact output wording can vary slightly between MySQL versions.

7.1.1 Creating the User (Setup)

SQL Command and Output:

```
mysql> CREATE USER IF NOT EXISTS 'student_user'@'localhost'
    -> IDENTIFIED BY 'Student@123';
Query OK, 0 rows affected (0.013 sec)
```

7.1.2 GRANT Command

Definition: GRANT gives a user permission to perform an operation on a table. Here the user is allowed to read (SELECT) the Department table only.

SQL Command and Output:

```
mysql> GRANT SELECT ON StudentInformationSystem.Department
    -> TO 'student_user'@'localhost';
Query OK, 0 rows affected (0.010 sec)

mysql> SHOW GRANTS FOR 'student_user'@'localhost';
+-----------------------------------------------------------------------------------------+
| Grants for student_user@localhost                                                       |
+-----------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `student_user`@`localhost`                                        |
| GRANT SELECT ON `StudentInformationSystem`.`Department` TO `student_user`@`localhost` |
+-----------------------------------------------------------------------------------------+
2 rows in set (0.000 sec)
```

Explanation: The SHOW GRANTS output confirms that student_user can now run SELECT on the Department table. `USAGE` only means that the user can connect and has no other global privileges.

7.1.3 REVOKE Command

Definition: REVOKE removes a permission that was previously granted to a user.

SQL Command and Output:

```
mysql> REVOKE SELECT ON StudentInformationSystem.Department
    -> FROM 'student_user'@'localhost';
Query OK, 0 rows affected (0.009 sec)

mysql> SHOW GRANTS FOR 'student_user'@'localhost';
+-------------------------------------------------+
| Grants for student_user@localhost               |
+-------------------------------------------------+
| GRANT USAGE ON *.* TO `student_user`@`localhost` |
+-------------------------------------------------+
1 row in set (0.000 sec)
```

Explanation: After REVOKE only the USAGE privilege remains, so the user can no longer read the Department table.

7.2 Transaction Control Language (TCL): COMMIT, ROLLBACK and SAVEPOINT

Definition: A transaction is a group of SQL statements that is treated as a single unit of work: either all of its changes are saved or none of them are. TCL commands control the transaction and decide whether database changes are saved permanently or cancelled. This keeps the data consistent even if an error occurs halfway through a task. The examples assume a transactional storage engine such as InnoDB.

Command	Purpose
START TRANSACTION	Begins a new transaction; changes stay temporary until COMMIT or ROLLBACK.
COMMIT	Permanently saves all changes made in the current transaction.
ROLLBACK	Cancels the changes made in the current transaction before they are committed.
SAVEPOINT	Marks a point inside a transaction so that ROLLBACK TO can undo only the later changes.

7.2.1 COMMIT Command

Definition: COMMIT permanently saves the changes made in the current transaction. After COMMIT the changes cannot be rolled back.

SQL Command and Output:

```
mysql> USE StudentInformationSystem;
Database changed

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.000 sec)

mysql> UPDATE Department
    -> SET Office_Location = 'Block F'
    -> WHERE Department_ID = 1;
Query OK, 1 row affected (0.001 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.004 sec)

mysql> SELECT Department_ID, Office_Location
    -> FROM Department
    -> WHERE Department_ID = 1;
+---------------+-----------------+
| Department_ID | Office_Location |
+---------------+-----------------+
|             1 | Block F         |
+---------------+-----------------+
1 row in set (0.000 sec)
```

Explanation: The office location of department 1 was changed from Block A to Block F and COMMIT made the change permanent for all users.

7.2.2 ROLLBACK Command

Definition: ROLLBACK cancels the changes made in the current transaction before they are committed, restoring the data to its state at the start of the transaction.

SQL Command and Output:

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.000 sec)

mysql> UPDATE Department
    -> SET Office_Location = 'Block Z'
    -> WHERE Department_ID = 1;
Query OK, 1 row affected (0.001 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT Department_ID, Office_Location FROM Department WHERE Department_ID = 1;
+---------------+-----------------+
| Department_ID | Office_Location |
+---------------+-----------------+
|             1 | Block Z         |
+---------------+-----------------+
1 row in set (0.000 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.003 sec)

mysql> SELECT Department_ID, Office_Location FROM Department WHERE Department_ID = 1;
+---------------+-----------------+
| Department_ID | Office_Location |
+---------------+-----------------+
|             1 | Block F         |
+---------------+-----------------+
1 row in set (0.000 sec)
```

Explanation: Inside the transaction the temporary value Block Z is visible, but ROLLBACK cancels it. The committed value (Block F) from Section 7.2.1 is kept; only the uncommitted change was undone.

7.2.3 SAVEPOINT Command

Definition: SAVEPOINT sets a named point inside a transaction. ROLLBACK TO SAVEPOINT undoes only the changes made after that point, while earlier changes of the transaction stay pending until COMMIT.

SQL Command and Output:

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.000 sec)

mysql> UPDATE Department SET Office_Location = 'Block G' WHERE Department_ID = 2;
Query OK, 1 row affected (0.001 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SAVEPOINT sp1;
Query OK, 0 rows affected (0.000 sec)

mysql> UPDATE Department SET Office_Location = 'Block H' WHERE Department_ID = 3;
Query OK, 1 row affected (0.001 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> ROLLBACK TO SAVEPOINT sp1;
Query OK, 0 rows affected (0.001 sec)
```

```
mysql> COMMIT;
Query OK, 0 rows affected (0.003 sec)

mysql> SELECT Department_ID, Office_Location
    -> FROM Department
    -> WHERE Department_ID IN (2, 3);
+---------------+-----------------+
| Department_ID | Office_Location |
+---------------+-----------------+
|             2 | Block G         |
|             3 | Block C         |
+---------------+-----------------+
2 rows in set (0.000 sec)
```

Explanation: The change to department 3 (Block H) was made after sp1 and was undone by ROLLBACK TO SAVEPOINT, while the change to department 2 (Block G) was kept and saved by COMMIT. DDL statements such as CREATE or DROP TABLE cause an implicit commit in MySQL, so they cannot be rolled back.

7.3 Summary of Command Categories

Category	Full form	Commands used in this project
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE
DCL	Data Control Language	GRANT, REVOKE
TCL	Transaction Control Language	START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT

8. Testing and Results

Every command in this report was executed in the MySQL client and its output compared with the expected result. The table below summarises the main tests and the results obtained.

No.	Feature tested	Test performed	Result
1	Database and tables	CREATE DATABASE and 10 CREATE TABLE statements; INSERT of sample data	Database created; 5 rows verified in each table
2	DDL	ALTER (add Address), DROP TABLE, TRUNCATE TABLE on a backup copy	Column added (7 columns); table dropped; Attendance empty
3	DML	INSERT, UPDATE and DELETE on Student	1 row affected each; changes verified with SELECT
4	Aggregate functions	COUNT, SUM, AVG, MIN, MAX on Marks	5 records, total 413, average 82.60, min 72, max 90
5	Joins	INNER, LEFT, RIGHT, FULL OUTER (UNION), CROSS	5, 5, 7, 7 and 25 rows respectively
6	GROUP BY / HAVING	Students per department; courses with average above 80	Counts 2, 2, 1, 0, 0; HAVING returned 2 groups
7	Nested queries	Subqueries on department, designation, marks and attendance	2, 2, 3 and 2 rows returned
8	Stored procedures	Six procedures created and called	Correct result set or OUT value for every CALL
9	Triggers (validation)	Marks of 150 inserted and 120 updated; pending fee deleted	All three blocked with ERROR 1644; data unchanged
10	Triggers (audit)	Student inserted, updated and deleted	3 audit rows created automatically
11	Views	Five views created and queried; update through CS_Students	Correct rows returned; base table updated
12	View restrictions	INSERT outside CHECK OPTION; UPDATE on aggregate view	ERROR 1369 and ERROR 1288 raised as expected
13	DCL	GRANT SELECT then REVOKE SELECT for student_user	SHOW GRANTS showed the privilege added and then removed
14	TCL	COMMIT, ROLLBACK and ROLLBACK TO SAVEPOINT	Committed changes persisted; uncommitted changes cancelled

All tests produced the expected results, which shows that the database design, constraints, views, procedures, triggers and access/transaction controls work together correctly for the sample data.

9. Conclusion

The Student Information System project shows how a relational database can organize student-related academic information in an institution. A ten-entity schema was designed with an ER diagram, implemented in MySQL with primary keys, foreign keys and constraints, and populated with sample data. SQL operations (DDL, DML, joins, aggregates and nested queries), views, stored procedures and triggers were used to retrieve, summarise, protect and audit the data, while DCL and TCL commands demonstrated access control and reliable transaction management.

Future scope: the database can be extended with a web or mobile front end, a secure login module that uses the UserAccount table, online fee payment, biometric attendance, and indexes and reports for larger volumes of data.